

# The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Review and Mapping Study

AMR MOHAMED, Queen's University, Canada

MARAM ASSI, Université du Québec à Montréal, Canada

MARIAM GUIZANI, Queen's University, Canada

Large language model assistants (LLM-assistants) present new opportunities to transform software development. Developers are increasingly adopting these tools across tasks, including coding, testing, debugging, documentation, and design. Yet, despite growing interest, there is no synthesis of how LLM-assistants affect software developer productivity. In this paper, we present a systematic review and mapping of 39 peer-reviewed studies published between January 2014 and December 2024 that examine this impact. Our analysis reveals that the majority of studies report considerable benefits from LLM-assistants, though a notable subset identifies critical risks. Commonly reported gains include accelerated development, minimized code search, and the automation of trivial and repetitive tasks. However, studies also highlight concerns around cognitive offloading and reduced team collaboration. Our study reveals that whether LLM-based assistants improve or degrade code quality remains unresolved, as existing studies report contradictory outcomes contingent on context and evaluation criteria. While the majority of studies (90%) adopt a multi-dimensional perspective by examining at least two SPACE dimensions, reflecting increased awareness of the complexity of developer productivity, only 15% extend beyond three dimensions, indicating substantial room for more integrated evaluations. Satisfaction, Performance, and Efficiency are the most frequently investigated dimensions, whereas Communication and Activity remain underexplored. Most studies are exploratory (59%) and methodologically diverse, but lack longitudinal and team-based evaluations. This review surfaces key research gaps and provides recommendations for future research and practice. All artifacts associated with this study are publicly available at <https://zenodo.org/records/18489222>.

CCS Concepts: • **Software and its engineering** → **Software creation and management**; • **Human-centered computing**;

Additional Key Words and Phrases: Software Engineering, Developer Productivity, AI Assistants, Large Language Model, LLM4SE

## 1 Introduction

Large Language Models (LLMs) are increasingly being integrated into the software engineering (SE) domain [1]. In particular, the emergence of LLM-assistants, the term we use to refer to generative AI tools powered by LLMs that support software development tasks, has driven rapid adoption in both research and practice. Examples include OpenAI's GPT-series (e.g., GPT-4 [2]) and GitHub Copilot [3], which are now commonly used to assist with tasks such as code generation and completion [4, 5, 6], code translation [7, 8], debugging and maintenance [9, 10, 11], documentation [12, 13], and system design [14, 15]. These tools support a new development paradigm often referred to as AI pair programming, in which developers interactively engage with LLM-assistants throughout the

---

Authors' addresses: Amr Mohamed, [amr.m@queensu.ca](mailto:amr.m@queensu.ca), Queen's University, Kingston, ON, Canada; Maram Assi, [assi.maram@uqam.ca](mailto:assi.maram@uqam.ca), Université du Québec à Montréal, Montréal, QC, Canada; Mariam Guizani, [mariam.guizani@queensu.ca](mailto:mariam.guizani@queensu.ca), Queen's University, Kingston, ON, Canada.

---

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/4-ART

<https://doi.org/10.1145/3809494>

software development process [16]. Since the public release of ChatGPT<sup>1</sup> in late 2022, an expanding ecosystem of LLM-powered coding assistants—such as Cursor<sup>2</sup>, Windsurf<sup>3</sup>, and Bolt<sup>4</sup> has emerged. This widespread integration underscores the growing reliance on LLMs in SE and raises critical questions about their impact on software developer productivity.

Software developer productivity is a multifaceted construct that encompasses not only the efficiency and quality of software production but also the satisfaction, collaboration, and cognitive load experienced by a developer. While early approaches for measuring productivity relied on quantifiable outputs such as lines of code (LOC) or development velocity [17, 18], recent research highlights the importance of human-centered factors such as communication, satisfaction, and well-being [19]. As LLM-assistants become increasingly integrated into development workflows, it is crucial to understand their impact on software developer productivity.

To address this gap, we conduct a systematic review and mapping of 39 peer-reviewed studies published between 2014 and December 2024 that examine the impact of LLM-assistants on software developer productivity. Our review analyzes methodological strategies, evaluation practices, and the productivity dimensions these studies engage with. We synthesize reported benefits and risks, and apply established conceptual frameworks to map our findings and contextualize their broader implications. Based on this synthesis, we identify key research gaps and provide actionable recommendations for both researchers and practitioners. This paper makes the following contributions:

- We present the first systematic review and mapping of the literature focused on the impact of LLM-assistants on software developer productivity, synthesizing evidence from 39 peer-reviewed primary studies published between 2014 and December 2024.
- We provide a structured characterization of the methodological strategies and evaluation practices used to assess developer productivity, and synthesize the reported effects of LLM-assistants, surfacing key benefits (e.g., reduced task initiation overhead, support for code-adjacent tasks) and risks (e.g., over-reliance, flow disruption).
- We analyze our findings through the lens of the *SPACE* framework and employ McLuhan’s Tetrad framework in our discussion to reflect on broader socio-technical implications.
- We offer actionable recommendations for practitioners and researchers, and release a publicly available replication package [20] containing all study data, selection decisions, and exclusion rationales to support transparency and reproducibility.

The remainder of this paper is structured as follows. Section 2 provides background on software developer productivity. Section 3 outlines the methodology used to conduct our review, including search strategies, selection criteria, and data extraction procedures. Sections 4 through 7 focus on addressing each of the research questions separately. Section 8 discusses implications, recommendations for practitioners, and directions for future research. Section 9 examines the threats to validity. Finally, section 10 concludes the paper.

## 2 Background

Software developer productivity has received sustained research attention since the early decades of software engineering. Notably, Frederick J Brooks noted in his 1975 book *The Mythical Man Month* [21] that there is “no silver bullet” for improving productivity, underscoring that software development is a complex, human-centric activity not easily optimized by simply adding more resources.

<sup>1</sup><https://chatgpt.com/>

<sup>2</sup><https://www.cursor.com/>

<sup>3</sup><https://www.windsurf.com/>

<sup>4</sup><https://bolt.new/>

This foundational perspective was further cemented by Tom DeMarco and Timothy Lister’s seminal work, *Peopleware: Productive Projects and Teams* [22], which established that the biggest obstacles to software productivity are not technical, but rather sociological and organizational. The authors argue that improving developer performance requires managing people, environment, and culture, not just tools or processes. Similarly, work by Grinter et al. [23] showed that the way software is architected and how teams are structured are deeply interconnected, misalignment between the two creates communication bottlenecks and coordination overhead. As projects scale or become divided across modules and teams, informal communication that once happened naturally becomes harder to sustain, requiring deliberate mechanisms for information sharing and decision tracking, illustrating that productivity is as much a social and organizational phenomenon as it is technical.

Since then, a wide range of studies have attempted to define and measure developer productivity [24, 25, 26, 27]. Despite decades of research, developer productivity remains a complex construct with no universal definition or measurement consensus. This ambiguity stems from the diverse variables that influence productivity [17]. Historically, software developer productivity has often been quantified using simple output-to-input ratios such as lines of code (LOC) [24, 28] or task completion time [29]. While these measures provide quantifiable proxies, they fail to capture the broader human, social, and organizational dimensions.

Recent research has shifted toward multidimensional frameworks to assess software developer productivity more comprehensively. This shift reflects the evolution of industry thinking toward continuous delivery and organizational performance [30], which links technical capabilities, team culture, and delivery performance. Noda et al. [31] propose a model based on Developer Experience (*DevEx*) [32], comprising three core dimensions: feedback loops, cognitive load, and flow state. Their work synthesizes insights from software engineering and human-computer interaction to operationalize how development environments and processes shape developers’ day-to-day experiences. The *DevEx* model offers a practical framework for assessing and improving productivity from the developer’s point of view. Forsgren et al. [19] introduce the *SPACE* framework, which characterizes software developer productivity across five dimensions: Satisfaction and well-being, Performance, Activity, Communication and collaboration, Efficiency and flow. Satisfaction and well-being refer to how fulfilled and healthy developers feel in relation to their work, tools, team, and organizational culture. Performance reflects the quality and effectiveness of the outcomes of a system or process, such as software reliability, absence of bugs, or customer satisfaction. Activity captures the count of observable work events or software artifacts such as commits, pull requests, code reviews, builds, or deployments. Communication and collaboration address how individuals and teams communicate, coordinate, and integrate their work through metrics like discoverability of documentation and expertise. Efficiency and flow focus on the uninterrupted progress of work at both individual and system levels. Consequently, the *SPACE* framework has been increasingly used in empirical studies to offer a more nuanced lens on productivity, especially in collaborative settings such as AI-assisted development [33, 34, 35, 36, 37, 38, 39].

The suggested multidimensional perspectives highlight that a single metric cannot meaningfully capture productivity and instead encourage the use of composite measures that reflect the complex and varied nature of software development work, including human-centric metrics.

### 3 Systematic Literature Review Methodology

We aim to establish the current state of evidence on the effects of LLM-assistants on software developer productivity. To this end, we conduct a comprehensive analysis across three key dimensions: (1) the methodological strategies, procedures, and instruments employed in primary studies (2) the reported benefits and risks associated with the use of LLM-based assistance, and (3) the specific dimensions of developer productivity that have been investigated. Our overarching objective is to synthesize the fragmented body of existing knowledge, highlight

methodological strategies and their instrumentation, and identify critical gaps to guide future research in this rapidly evolving area.

We ground our methodology in the seminal guidelines by Kitchenham and Charters [40], which are derived from evidence-based practices in medical research and have been adapted for use in SE.

In particular, this review is guided by the following research questions:

**RQ0: What are the characteristics of peer-reviewed studies that investigate the impact of LLM-assistants on software developer productivity?**

In RQ0, we contextualize the emerging research landscape surrounding LLM-assisted software development. Specifically, we examine this landscape from three angles: (1) an overview of the temporal distribution of publications, (2) the publication venues and their disciplinary focus, and (3) the patterns of authorship across the research community.

**RQ1: What are the methodological strategies, procedures, and instruments used by peer-reviewed studies that investigate the impact of LLM-assistants on software developer productivity?**

In RQ1, we examine how existing research is conducted to investigate the relationship between LLM-assistants and software developer productivity. Specifically, we investigate the empirical strategies adopted to study the impact of LLM-assistants, (2) the procedures and study designs used to carry out these investigations, and (3) the instruments and metrics employed to evaluate software developer productivity.

**RQ2: What is the impact of LLM-assistants on software developer productivity?**

In RQ2, we explore the effects of LLM-assistants on software development practices. First, we summarize the overall findings from the identified primary studies. We supplement these findings by analyzing the reported benefits and risks of using LLM-assistants across diverse study settings. This question aims to provide a structured understanding of how these tools affect developers in practice and what trade-offs they introduce.

**RQ3: Which dimensions of developer productivity are investigated and how do these dimensions map onto the SPACE framework?**

In RQ3, we investigate how productivity is defined and assessed in the context of LLM-assisted software development. To provide additional insights, we map the main focus of each study to the dimensions of the *SPACE* framework, i.e., Satisfaction and well-being, Performance, Activity, Communication and collaboration, and Efficiency and flow. This analysis offers a clear understanding of how the concept of productivity is operationalized across the existing body of work and highlights underexplored dimensions to guide future research.

### 3.1 Pre-review mapping

We adhere to the guidelines proposed by Kitchenham and Charters [40] for piloting the research protocol through pre-review mapping. To plan the construction of our review, we conducted a pre-review planning study that involved defining the research questions, establishing the inclusion and exclusion criteria, identifying a set of control papers, and iteratively refining the search strings. Detailed steps of this pre-review mapping (including the identification of control papers and query refinement) are all provided in the supplemental appendix [20]

**3.1.1 Control papers identification.** Once our research questions defined, we set our list of inclusion and exclusion criteria (see section 3.1.1). We then performed a pilot search to identify a small set of control papers against which potential search string could be validated. This pilot involved manually searching for relevant publications, screening titles and abstracts, and conducting one round of backward and forward snowballing. The process

yielded 17 control papers that met our criteria. These control papers, whose selection details are reported in the supplemental appendix [20], were subsequently used to validate the search string.

Table 1. Database search strings and results. Total n = 9,756.

| Database       | Search String                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Results (since 2014) |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| ACM            | (Language Model* OR "LM" OR "LMs" OR "LLM" OR "LLMs" OR "Artificial Intelligence" OR "AI") AND ((title:(Software Engineer* OR Software Develop* OR Developer* OR Coder* OR Programmer*)) OR (abstract: (Software Engineer* OR Software Develop* OR Developer* OR Coder* OR Programmer*))) AND (Productivity)                                                                                                                                                   | 4,044                |
| IEEE Xplore    | ((Language Model OR "LM" OR "LMs" OR "LLM" OR "LLMs" OR "Artificial Intelligence" OR "AI") AND ((("Document Title":Software Engineer OR "Document Title":Software Develop* OR "Document Title":Developer OR "Document Title":Coder OR "Document Title":Programmer) NEAR/5 (Productivity)) OR ((("Abstract":Software Engineer OR "Abstract":Software Develop* OR "Abstract":Developer OR "Abstract":Coder OR "Abstract":Programmer) NEAR/5 (Productivity))))    | 491                  |
| ScienceDirect  | ((Language Model OR "LM" OR "LMs" OR "LLM" OR "LLMs" OR "Artificial Intelligence" OR "AI") AND (Productivity))<br>Advanced Search: Title, abstract, keywords: (Software Engineer OR Software Development OR Developer OR Coder OR Programmer)                                                                                                                                                                                                                  | 3,734                |
| Web of Science | ALL=(Language Model OR "LM" OR "LMs" OR "LLM" OR "LLMs" OR "Artificial Intelligence" OR "AI") AND ((TI=(Software Engineer NEAR Productivity OR Software Develop* NEAR Productivity OR Developer NEAR Productivity OR Coder NEAR Productivity OR Programmer NEAR Productivity)) OR (AB=(Software Engineer NEAR Productivity OR Software Develop* NEAR Productivity OR Developer NEAR Productivity OR Coder NEAR Productivity OR Programmer NEAR Productivity))) | 271                  |
| Scopus         | ALL( LANGUAGE Model* OR "LM" OR "LMs" OR "LLM" OR "LLMs" OR "Artificial Intelligence" OR AI ) AND TITLE-ABS ( ( Software Engineer* W/5 Productivity ) OR (Software Develop* W/5 Productivity ) OR ( Developer* W/5 Productivity ) OR ( Coder* W/5 Productivity ) OR ( Programmer* W/5 Productivity ) )                                                                                                                                                         | 836                  |
| Springer       | (Language Model* OR "LM" OR "LMs" OR "LLM" OR "LLMs" OR "Artificial Intelligence" OR "AI") AND "Productivity" Title : (Software Engineer* OR Software Develop* OR Developer* OR Coder* OR Programmer*)                                                                                                                                                                                                                                                         | 380                  |

**Inclusion and Exclusion Criteria.** We select papers for our study based on the following inclusion (IC) and exclusion (EC) criteria. The supplemental material [20] includes the exclusion decision for each study based on specific IC & EC criteria.

**Inclusion (IC):**

- (+) IC1: The paper investigates the effect of AI or LLMs on software developer productivity.
- (+) IC2: The paper is in English.
- (+) IC3: The paper has an accessible full text and was published in 2014 or later.

**Exclusion (EC):**

- (–) EC1 (Out of Scope): The paper does not focus on SE or does not explore the impact of AI or LLMs on software developer productivity.
- (–) EC2 (Out of Focus): The paper mentions the impact of AI or LLM on software developer productivity without it being one of the topics of the study.
- (–) EC3 (Publication Type): The paper belongs to any of the following categories: secondary studies; work-in-progress, extended abstracts, posters, tool demos, editorials, or grey literature; studies published in books, theses, workshop, monographs, keynotes, panels, doctoral symposium, or any other venues without a formal peer-review process.

- (–) EC4 (Length): The paper is a short publication with fewer than four pages.
- (–) EC5 (Accessibility): The full text of the paper is not accessible online (e.g., behind paywalls without institutional access, unavailable PDFs, or inaccessible publisher archives).

**3.1.2 Query formulation and refinement.** Following the guidelines proposed by Kitchenham and Charters [40], we selected six major digital libraries widely used in software engineering research: ACM Digital Library<sup>5</sup>, IEEE Xplore<sup>6</sup>, ScienceDirect<sup>7</sup>, Web of Science<sup>8</sup>, SpringerLink<sup>9</sup>, and Scopus<sup>10</sup>.

Constructing effective search queries is a challenging step in developing the review protocol, particularly due to the absence of standardized guidelines for selecting search terms [40]. This process relies on iterative refinement [41], especially in contexts where terminologies such as those used to describe AI and LLM in SE vary significantly between studies. Given the growing volume of research in this area, we observe that overly broad queries return a large number of false positives, thereby reducing the precision of the search query. Conversely, narrow queries risk omitting relevant studies [42]. We iteratively developed and refined the search string by validating candidate queries against the control papers, adjusting keywords and query structure as needed to balance precision and recall. After five query iterations, all authors held a consensus meeting and agreed on the final search string (see Table 1). The final selected search query successfully retrieves all 17 control papers.

Table 1 presents the finalized search query used in our review. Each search query consists of three segments separated by the AND string. The first segment related to AI or LLMs limits the filter to the AI or LLM technology. The second segment refers to the actor (i.e., software developers), and the third segment captures the concept in question (i.e., productivity).

In alignment with recent SLRs [43, 44, 45], we restrict our searches to the title, abstract, and keywords, as this improves precision. We also leverage proximity operators in the query, namely “NEAR/5” or “w/5” in IEEE Xplore and Scopus respectively, to improve the contextual relevance of matched terms [46]. This operator retrieves documents where the specified terms appear within five words of each other. This refinement is only applied in IEEE Xplore, Web of Science and Scopus, as the ACM Digital Library, Springer and ScienceDirect do not support it.

## 3.2 Primary Study Selection Process

We execute the search protocol as illustrated in Figure 1. First, we apply the finalized search strings on the selected digital libraries, restricting the results to publications from 2014 onward. This yields an initial set of ( $n = 9,756$ ) records, including 4,044 papers from ACM, 491 papers from IEEE Xplore, 3,734 papers from ScienceDirect, 271 papers from Web of Science, 836 papers from Scopus, and 380 papers from Springer. After removing duplicates, we obtain a set of 8,953 unique records ( $n = 803$  excluded). The first author performs an initial screening of all records based on their titles and abstracts, a process that took a total of 47 days. We use the tool *Rayyan*<sup>11</sup> to tag any excluded paper with the corresponding exclusion criteria (the list of all excluded papers and their corresponding exclusion criteria is provided in the supplemental material [20]). The second and last authors independently validate the excluded papers. Three meetings were held to discuss any disagreements until reaching a consensus. We adopt a conservative screening approach whereby records with insufficient information in the title and abstract to support a clear inclusion or exclusion decision were included in full-text review. The title and abstract screening process resulted in 228 papers (see Figure 1) with a total of 8,725 excluded papers.

<sup>5</sup><https://dl.acm.org/>

<sup>6</sup><https://ieeexplore.ieee.org/>

<sup>7</sup><https://www.sciencedirect.com/>

<sup>8</sup><https://www.webofscience.com/>

<sup>9</sup><https://link.springer.com/>

<sup>10</sup><https://www.scopus.com/>

<sup>11</sup><https://rayyan.ai/>

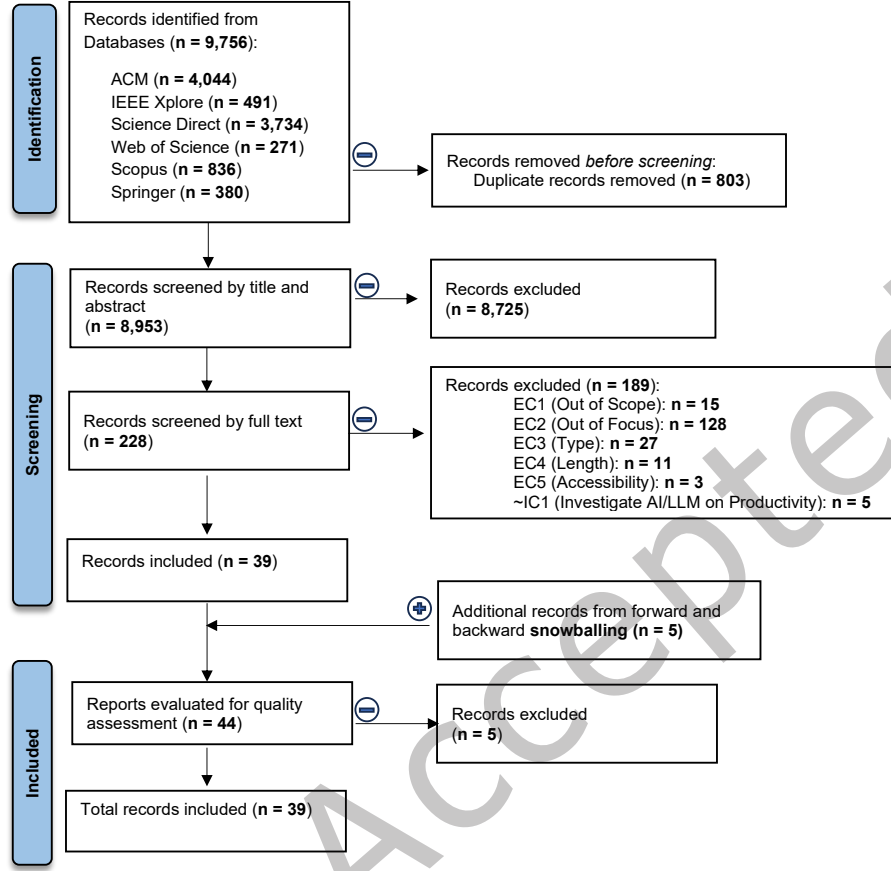


Fig. 1. Overview of the selection process for primary studies included in the review using PRISMA flow chart [47].

The full-text screening phase was conducted over a period of 10 weeks. This phase involves the careful reading and evaluation of the remaining 228 papers, applying the inclusion and exclusion criteria to assess their eligibility. For a detailed rationale on the inclusion and exclusion criteria during full-text screening, please refer to the supplemental material [20]. Throughout this process, the first author led the full-text screening of papers. Whenever the relevance of a study was unclear, it was flagged and reviewed in consultation with the second and last authors to ensure alignment with the protocol. This results in the exclusion of 189 studies during the full-text screening process.

To further expand our study set, we conduct a snowballing procedure by examining both the references and citations of the 39 selected studies. This phase took approximately two weeks, resulting in the identification and inclusion of five additional articles, expanding the set to 44 primary studies.

### 3.3 Quality Assessment

To ensure the reliability and rigor of the evidence synthesized in this review, we assessed the quality of the selected primary studies. We adopted the quality assessment strategy defined by Lenarduzzi et al. [48], which provides a comprehensive framework for evaluating empirical software engineering studies.

We evaluated each study against 11 criteria (QA1–QA11) designed to assess the clarity of research aims, the appropriateness of the methodology, the rigor of data analysis, and the validity of the findings. Table 2 details the specific criteria used.

Table 2. Quality Assessment Criteria (adapted from Lenarduzzi et al. [48])

| ID   | Quality Assessment Criterion                                                                         |
|------|------------------------------------------------------------------------------------------------------|
| QA1  | Is the paper based on research (or is it merely a “lessons learned” report based on expert opinion)? |
| QA2  | Is there a clear statement of the aims of the research?                                              |
| QA3  | Is there an adequate description of the context in which the research was carried out?               |
| QA4  | Was the research design appropriate to address the aims of the research?                             |
| QA5  | Was the recruitment strategy appropriate for the aims of the research?                               |
| QA6  | Was there a control group with which to compare treatments?                                          |
| QA7  | Was the data collected in a way that addressed the research issue?                                   |
| QA8  | Was the data analysis sufficiently rigorous?                                                         |
| QA9  | Has the relationship between researcher and participants been considered to an adequate degree?      |
| QA10 | Is there a clear statement of findings?                                                              |
| QA11 | Is the study of value for research or practice?                                                      |

Following the scoring procedure proposed in [48], each criterion was graded on a five-point Likert scale: *Excellent* (4), *Very Good* (3), *Good* (2), *Fair* (1), and *Poor* (0). The detailed quality scores for each primary study are provided in the supplemental appendix [20]. Studies that failed to meet a minimum quality threshold of 50% average score [49] were excluded from the final set. This process resulted in the exclusion of 5 studies, leaving a final set of 39 primary studies. The majority of the studies were rated above 3. Detailed results of the quality assessment are available in the replication package material [20].

### 3.4 Data Extraction and Synthesis

In the final stage of the review process, the synthesis of findings for RQs across these studies were conducted over a period of three months. we followed a qualitative synthesis approach, consistent with Kitchenham’s guidelines [40]. To enable systematic integration, we first perform an initial thematic analysis iteration over all primary studies to extract relevant data (e.g., study goals, tools, empirical strategy and design, tasks, settings, and key results). In parallel, for each study, we write a descriptive summary. Then, our synthesis proceeded in multiple thematic analysis iterations. We first perform an initial thematic analysis phase to capture methodological details and observations. Then we follow up with three targeted iterations to capture methodological details (RQ1), synthesize benefits and risks (RQ2), and map studies to the SPACE framework (RQ3). After themes were consolidated, the first and last authors jointly validated the synthesized findings by cross-checking citations against the original text to ensure accuracy and traceability.

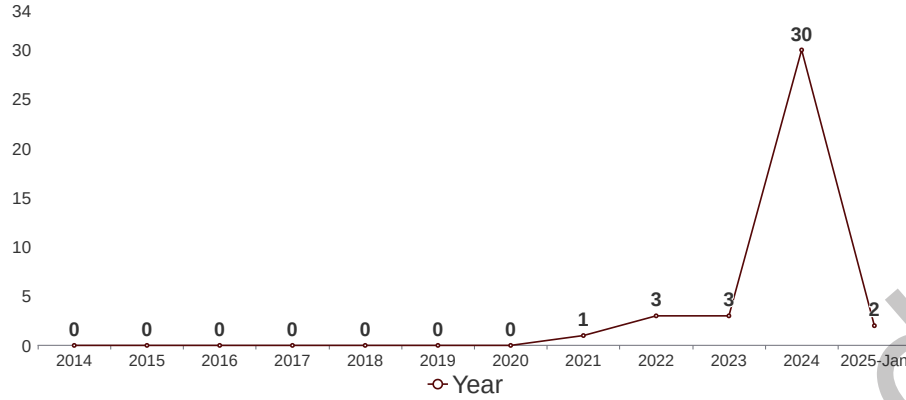


Fig. 2. Publication frequency per year.

#### 4 RQ0: What are the characteristics of peer-reviewed studies that investigate the impact of LLM-assistants on software developer productivity?

##### 4.1 Publication years

Figure 2 shows the frequency of publications by year. Although we included studies published more than ten years preceding our review, only four were published between 2014 and 2022 [PS1, PS2, PS3, PS4]. Research interest began to rise in 2022, which coincides with the release of ChatGPT<sup>12</sup>. This culminated in a sharp peak in 2024, which accounts for 77% of all included studies, likely due to increased accessibility and interest following ChatGPT's release.

##### 4.2 Author distributions

We investigate the distribution of papers per author across the 154 authors of primary studies. Most authors (i.e., 147) have a single publication, while 6 authors have two papers, and the most prolific author, Igor Steinmacher, has three publications. This distribution is probably due to the fact that the investigation of the impact of LLM-assistants on software developer is a relatively new topic that is just starting to build momentum.

##### 4.3 Publication venues

We extract the publication venue for each primary study. Table 3 shows the venues categorized by research focus. The majority (46%) of primary studies fall under "Software Engineering and Computer Science" published in venues including TOSEM, ICSE, and EASE. Human-Computer Interaction (HCI) is the second most prominent research focus with 18% (7 out of 39) primary studies published in venues such as CHI and IUI. The remainder of the primary studies are similarly distributed among specialized venues focusing on AI for Software Engineering / AI Engineering, Software Engineering Education, Information Systems and Decision Science, and Human-Aspects and Socio-Economic Impact. The variety of publication venues highlights the breadth and depth of the topic under study. The integration of LLM-assistants into the software development workflow introduces important considerations related to usability, automation, interaction design, and developer behavior.

<sup>12</sup>ChatGPT: <https://openai.com/chatgpt>

Table 3. Distribution of primary studies by publication venues.

| Research Focus                               | Venue                                                                                                           | Primary Studies      | %   |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------|-----|
| Software Engineering and Computer Science    | Proceedings of the ACM on Software Engineering (PACMSE)                                                         | [PS5, PS6, PS7, PS8] | 46% |
|                                              | ACM Transactions on Software Engineering and Methodology (TOSEM)                                                | [PS1, PS9, PS10]     |     |
|                                              | International Conference on Software Engineering (ICSE)                                                         | [PS11, PS12, PS13]   |     |
|                                              | Software Engineering in Practice (ICSE-SEIP)                                                                    | [PS14]               |     |
|                                              | ACM International Conference on the Foundations of Software Engineering (FSE)                                   | [PS15]               |     |
|                                              | ACM SIGPLAN International Symposium on Machine Programming (PLDI)                                               | [PS16]               |     |
|                                              | Automated Software Engineering (ASE)                                                                            | [PS17]               |     |
|                                              | Software Quality, Reliability, and Security Companion (QRS-C)                                                   | [PS18]               |     |
|                                              | Science of Computer Programming                                                                                 | [PS19]               |     |
|                                              | Evaluation and Assessment in Software Engineering (EASE)                                                        | [PS20]               |     |
|                                              | International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)                      | [PS21]               |     |
| Human-Computer Interaction (HCI)             | ACM Conference on Human Factors in Computing Systems (CHI)                                                      | [PS3, PS22]          | 18% |
|                                              | International Conference on Intelligent User Interfaces (IUI)                                                   | [PS2, PS23]          |     |
|                                              | Proceedings of the ACM on Human-Computer Interaction (CSCW)                                                     | [PS24]               |     |
|                                              | ACM Transactions on Computer-Human Interaction (TOCHI)                                                          | [PS25]               |     |
|                                              | Topics in Cognitive Science                                                                                     | [PS4]                |     |
| Information Systems and Decision Science     | Journal of Decision Systems (JDS)                                                                               | [PS26]               | 13% |
|                                              | Proceedings of the Americas Conference on Information Systems (AMCIS)                                           | [PS27]               |     |
|                                              | Innovations in Software Engineering Conference (ISEC)                                                           | [PS28]               |     |
|                                              | International Conference on Decision Aid Sciences and Applications (DASA)                                       | [PS29]               |     |
|                                              | Hawaii International Conference on System Sciences (HICSS)                                                      | [PS30]               |     |
| Human-Aspects and Socio-Economic Impact      | Futures                                                                                                         | [PS31]               | 10% |
|                                              | Structural Change and Economic Dynamics                                                                         | [PS32]               |     |
|                                              | Cooperative and Human Aspects of Software Engineering (CHASE)                                                   | [PS33]               |     |
|                                              | Annual Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT) | [PS34]               |     |
| AI for Software / Engineering AI Engineering | AI Engineering - Software Engineering for AI (IEEE/ACM)                                                         | [PS35]               | 8%  |
|                                              | AI-Powered Software (AIware)                                                                                    | [PS36]               |     |
|                                              | International Conference on Generative Artificial Intelligence and Information Security (GAIIS)                 | [PS37]               |     |
| Software Engineering Education               | Innovation and Technology in Computer Science Education (ITiCSE)                                                | [PS38]               | 5%  |
|                                              | ICSE Software Engineering Education and Training (ICSE-SEET)                                                    | [PS39]               |     |

#### 4.4 Most frequently used LLM tools

Table 4 summarizes all LLM-assistants used across the primary studies. The most frequently evaluated tools are ChatGPT (15 studies), Github Copilot (14 studies) and followed by Tabnine (3 studies), GPT-4 (3 studies), CodeWhisperer (3 studies) and GPT-3.5 (2 studies). All Other tools were used only once, such as Claude, and Codex.

Table 4. Summary of all LLM tools used in the primary studies and their associated study IDs.

| Tool                       | Count | Primary studies                                                                          |
|----------------------------|-------|------------------------------------------------------------------------------------------|
| ChatGPT                    | 15    | [PS7, PS8, PS11, PS13, PS14, PS15, PS19, PS20, PS21, PS28, PS29, PS33, PS34, PS36, PS39] |
| GitHub Copilot             | 14    | [PS10, PS11, PS14, PS15, PS16, PS22, PS24, PS27, PS29, PS30, PS33, PS34, PS36, PS38]     |
| Tabnine                    | 3     | [PS10, PS11, PS15]                                                                       |
| GPT-4                      | 3     | [PS12, PS21, PS29]                                                                       |
| CodeWhisperer              | 3     | [PS10, PS11, PS14]                                                                       |
| GPT-3.5                    | 2     | [PS21, PS24]                                                                             |
| Claude                     | 1     | [PS15]                                                                                   |
| Codex                      | 1     | [PS25]                                                                                   |
| Gemini                     | 1     | [PS15]                                                                                   |
| GPT-3                      | 1     | [PS18]                                                                                   |
| Ansible Lightspeed         | 1     | [PS17]                                                                                   |
| Bard                       | 1     | [PS23]                                                                                   |
| CodeGen2 (7B)              | 1     | [PS10]                                                                                   |
| GILT                       | 1     | [PS13]                                                                                   |
| Internal tool: CodeCompose | 1     | [PS5]                                                                                    |
| NL2Code PyCharm plugin     | 1     | [PS1]                                                                                    |
| StackSpotAI                | 1     | [PS35]                                                                                   |
| StarCoder (7B)             | 1     | [PS10]                                                                                   |
| TransCoder                 | 1     | [PS2]                                                                                    |
| aiXcoder                   | 1     | [PS10]                                                                                   |
| OpenAI API                 | 1     | [PS36]                                                                                   |
| Midjourney                 | 1     | [PS36]                                                                                   |

**RQ0- Summary**

The majority of studies (35 out of 39) were published after the release of ChatGPT in November 2022. Only four earlier studies (2014–2022) precede this period and rely on pre-LLM paradigms. Most authors (147 out of 154) contributed a single publication, while seven authors published two or more papers. Most studies were published in Software Engineering and Computer Science venues (18 studies), followed by Human-Computer Interaction venues (7 studies).

- 5 RQ1: What are the methodological strategies, procedures, and instruments used by peer-reviewed studies that investigate the impact of LLM-assistants on software developer productivity?

### 5.1 Distribution of the research strategies

We classify the 39 primary studies based on Stol and Fitzgerald [50] taxonomy for empirical software engineering strategies (see Table 5), which is a taxonomy built to distinguish studies' strategies according to their level of obtrusiveness (e.g., control) and generalizability (e.g., realism). This classification offers a structured understanding of how the research community has approached the investigation of AI tools. Laboratory experiments are the most common strategy, used by 38% of the primary studies (15 out of 39). Laboratory experiments rely on controlled environments to isolate the effects of LLM-assistants on specific development tasks. Field studies are the second most common strategy, representing 23% (9 out of 39) of the primary studies. Field studies prioritize ecological

Table 5. Distribution of research strategies across the primary studies.

| Strategy                | Primary Study                                                                         | Percent |
|-------------------------|---------------------------------------------------------------------------------------|---------|
| Field Study             | [PS5, PS7, PS14, PS15, PS17, PS27, PS33, PS36, PS37]                                  | 23%     |
| Field Experiment        | [PS30, PS39]                                                                          | 5%      |
| Experimental Simulation | [PS19, PS21, PS22, PS28, PS29]                                                        | 13%     |
| Laboratory Experiment   | [PS1, PS2, PS3, PS6, PS8, PS10, PS12, PS13, PS18, PS20, PS23, PS24, PS25, PS35, PS38] | 38%     |
| Sample Study            | [PS4, PS9, PS11, PS16, PS26, PS32]                                                    | 15%     |
| Judgment Study          | [PS31, PS34]                                                                          | 5%      |

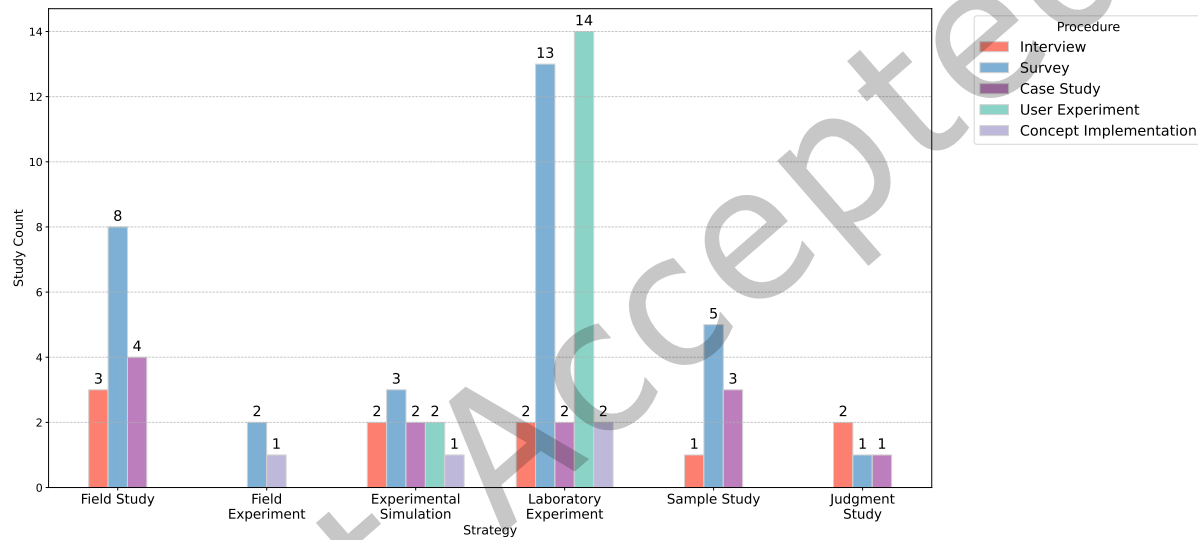


Fig. 3. The distribution of empirical procedures across methodological strategies.

validity by observing developer behavior in real-world settings without the need for researcher intervention. Sample studies, typically large-scale surveys, account for 15% (6 out of 39), they aim to capture broad trends and perceptions across diverse developer populations.

Other strategies are less frequent. Field experiments (5%, 2 out of 39) resemble field studies in that they occur in real-world settings. However, unlike field studies, researchers actively manipulate specific variables such as changes to a tool or process to evaluate their effects within the natural context. Experimental simulations (13%, 5 out of 39) combine elements from both laboratory and field studies by replicating real-world scenarios within controlled environments to study certain phenomena under realistic but simulated conditions. Judgment studies (5%, 2 out of 39) involve collecting expert opinions in a structured manner in a series of interviews and questionnaires.

Table 6. Methodological procedures used in the empirical studies.

| Procedure                                 | Primary Studies                                                                                                                                                                           | Percent |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Survey                                    | [PS1, PS2, PS3, PS5, PS7, PS8, PS9, PS11, PS12, PS13, PS14, PS15, PS16, PS17, PS18, PS19, PS20, PS21, PS22, PS23, PS24, PS25, PS26, PS27, PS30, PS31, PS32, PS35, PS36, PS37, PS38, PS39] | 82%     |
| User Experiment                           | [PS1, PS2, PS3, PS6, PS8, PS12, PS13, PS18, PS20, PS22, PS23, PS24, PS25, PS29, PS35, PS38]                                                                                               | 41%     |
| Concept Implementation (Proof of Concept) | [PS1, PS10, PS28, PS39]                                                                                                                                                                   | 10%     |
| Interview                                 | [PS3, PS15, PS21, PS25, PS26, PS29, PS31, PS33, PS34, PS37]                                                                                                                               | 26%     |
| Case Study                                | [PS2, PS3, PS4, PS14, PS16, PS21, PS26, PS27, PS28, PS34, PS36, PS37]                                                                                                                     | 31%     |

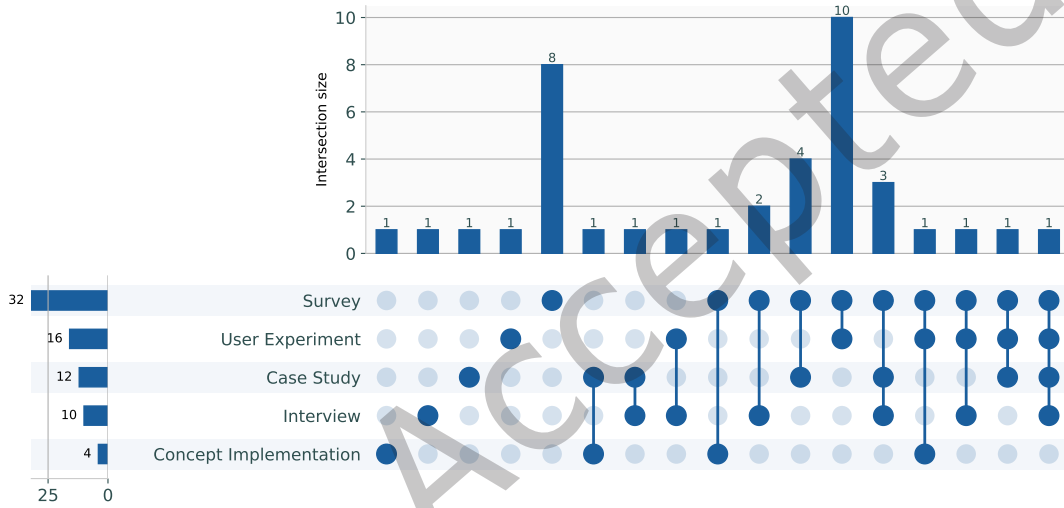


Fig. 4. Classification of methodological procedures classification and their overlap. Each bar on the left represents the number of studies that use a specific research method, while the bars along the top indicate how many studies employ a given combination of methods. For example, the most common pairing is a user experiment combined with a survey, used in 10 unique studies.

## 5.2 Distribution of the research procedures

We adapt a fine-grained taxonomy identified from prior literature Glass, Vessey, and Ramesh [51] to assign one or more methodology procedures to each primary study, such as interviews, surveys, or user experiments (i.e., controlled experiments, quasi-experiments), based on the primary methods used (see Table 6).

Among the primary studies, we find 90% (35 out of 39) of the studies leverage self-reported data such as surveys and interviews, and 41% of the studies (16 out of 39) are conducted in experimental settings (e.g., user studies, controlled experiments, or quasi-experiments). User experiments are almost exclusively associated with the laboratory experiment research strategy as shown in Figure 3. Additionally, 69% of these studies (27 out of 39) adopt a mixed-method approach as illustrated in Figure 4). For example, the most common combination of methods is a “user experiment” paired with a “survey”. This approach is frequently used to triangulate self-reported perceptions (e.g., user experience or satisfaction) with measured performance metrics.

To provide additional insight into the nature of the current research direction, we assess the empirical studies based on their objectives, adopting a taxonomy from Hartson et al. [52] to classify the objective of each study into one of two approaches: formative and summative (see supplemental material [20] for detailed classification). A study with a formative objective primarily focuses on exploring, refining, or improving a process, tool, or methodology. In contrast, a study with a summative objective focuses on drawing conclusions about the effectiveness, outcomes, or impact of a completed process, tool, or methodology. We find that 59% (23 out of 39) of the studies have a formative goal, and 41% (16 out of 39) have a summative goal. This demonstrates that the current research direction reflect a balanced research landscape, rather than a strong focus on final, conclusive outcome.

We also classify each empirical study based on its adopted methods of data analysis: quantitative, qualitative, or both (see supplemental material [20] for classification details). 67% (26 out of 39) of the empirical primary studies include a mix of quantitative and qualitative analysis, 21% (8 out of 39) of the studies rely only on qualitative analysis, and 13% (5 out of 39) of the studies rely only on quantitative analysis.

Table 7. Data sources and origin used by empirical studies.

| Data Source                                 | Instrument Origin                    | Instrument and Primary Studies                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Self-Reported</b>                        | Designed by Authors                  | Surveys [PS1, PS2, PS7, PS8, PS9, PS11, PS14, PS15, PS18, PS19, PS20, PS21, PS23, PS24, PS29, PS31, PS36, PS37, PS39]<br>Interviews [PS3, PS15, PS21, PS25, PS26, PS29, PS31, PS33, PS34, PS37]<br>Users open-ended feedback [PS5, PS17, PS25, PS35]                                                                                                                                                                                      |
|                                             | Validated Instruments and Frameworks | NASA-TLX (Mental Effort) [PS2, PS8, PS12, PS13, PS25, PS38]<br>SPACE Framework-Based Surveys [PS16, PS22, PS24, PS27]<br><br>Technology Acceptance Model (TAM) [PS9, PS13, PS39]<br>Self-Efficacy Questionnaires [PS3, PS12]<br>After-Action Review for AI (AAR/AI) [PS12]<br>Emotion Affect Questionnaire [PS38]                                                                                                                         |
| <b>Behavioral &amp; Performance Metrics</b> | Designed by Authors                  | Task Completion and Correctness [PS1, PS4, PS6, PS12, PS13, PS21, PS23, PS29, PS38]<br>Suggestions Acceptance Rate [PS5, PS6, PS10, PS16, PS17, PS22, PS24]<br>Interaction Patterns (Logs/Edits/Tracking) [PS1, PS8, PS13, PS17, PS23, PS24, PS25]<br>Time to Completion [PS1, PS3, PS4, PS6, PS8, PS13, PS18, PS23, PS24, PS25, PS27, PS28]<br>Code Quality Metrics [PS1, PS2, PS12, PS18, PS24, PS37]<br>Productivity Gain [PS28, PS29] |
|                                             | Validated Frameworks                 | Time Cost Quality (TCQ) Framework [PS32]<br>Resource-Based View (RBV) Framework [PS26, PS32]                                                                                                                                                                                                                                                                                                                                              |

### 5.3 Evaluation instruments

Researchers employ various instruments, from self-reported surveys and interviews including validated questionnaires to behavioral & performance metrics as shown in Table 7. Self-reported methods remain predominant, often designed by study authors to capture user experience, perceived productivity, trust, or ease of use (e.g., post-task surveys, open-ended feedback). Only a subset of the studies (15 out of 39) incorporate validated instruments including the *SPACE* framework [19], NASA-TLX for mental workload [53], TAM for technology acceptance [54], self-efficacy questionnaires [55, 56], or emotional affect questionnaire [57]. Behavioral & performance metrics focus on quantifiable outcomes, such as time to completion, acceptance rate of AI-generated suggestions, code quality metrics, and some analysis of interaction patterns (see Table 7). We find that behavioral & performance metrics are mostly associated with studies with a high level of control, such as laboratory experiments, field experiments, or experimental simulation (see Figure 5), for example, metrics such as time to completion or code quality metrics are mainly associated with laboratory experiments. In contrast, field studies and sample studies, while still employing diverse sets of instruments, primarily rely on self-reported methods, such as surveys, interviews, and users' open-ended feedback.

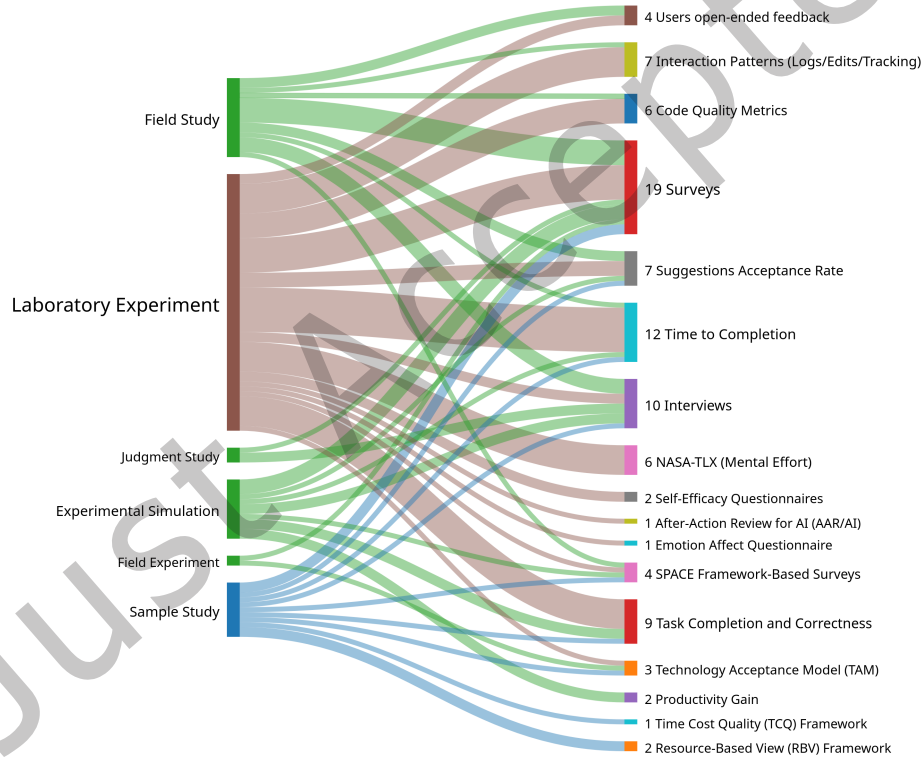


Fig. 5. Mapping between empirical strategies and instruments in primary studies.

**5.3.1 Time to completion.** Measuring the time required to complete tasks is the most frequently used performance metric. 31% (12 out of 39) of the empirical primary studies employ this measure. The majority of studies that measured task completion time are laboratory experiments [PS1, PS3, PS6, PS8, PS13, PS18, PS23, PS24, PS25],

which assess the time taken to complete specific programming tasks under controlled conditions. One field study conducted within a software company [PS27] measures time-related performance by comparing throughput and cycle time before and after the integration of Copilot. One experimental simulation [PS28] estimates total effort using a “person/days” metric to compare task completion durations with and without LLM-assistants. Finally, one study [PS4] measured completion time as the duration from when an issue was reported to when it was marked as resolved or closed.

**5.3.2 LLM suggestions acceptance rate.** Measuring the acceptance rate of LLM suggestions is one of the commonly used behavioral and performance metrics to measure productivity [PS5, PS6, PS10, PS16, PS17, PS22, PS24]. For instance, a study conducted by Meta [PS5] evaluates the adoption of an internal LLM-assistant coding system (Code Compose) within the company. The study quantifies the LLM-assistant’s utility by measuring both the number of LLM-generated suggestions accepted by developers and the proportion of code authored by the LLM-assistants. These metrics are then compared against reported acceptance rates from competing LLM-assistants to evaluate relative effectiveness.

One reason the acceptance rate metric is widely adopted is a study conducted by GitHub [PS16], which statistically analyzes the relationship between several interaction metrics related to code completion and developers’ self-reported productivity. The findings reveal a strong correlation between the frequency of accepted suggestions and perceived productivity. Despite these findings, the authors caution against using this metric in isolation to assess the effectiveness of LLM-assistants. They highlight that optimizing for acceptance rate may bias LLM-assistants toward well-represented languages or routine tasks, potentially disadvantaging less-represented workflows. Moreover, they warn that “blind” reliance on acceptance rate can lead to superficial improvements that inflate perceived usefulness without meaningfully enhancing developer outcomes.

**5.3.3 Mental effort and cognitive load.** Studies often use the terms mental effort or cognitive load interchangeably [58]. Reducing mental effort is considered a motivation for incorporating LLM-assistants in software development. Traditionally, studies aim to measure developer cognitive load through biometric modalities, such as electroencephalogram (EEG) or electrocardiogram (ECG) [58]. None of the identified studies leverages any biomedical measures or sensors for measuring cognitive load. Only one experimental study leverages eye-tracking [PS6] to measure the time participants spend reading code documentation.

Six studies (6 out of 39) [PS2, PS8, PS12, PS13, PS25, PS38] aim to measure cognitive workload by using NASA-TLX [53], which is a widely used questionnaire for assessing perceived mental workload. It captures six dimensions: mental demand, physical demand, temporal demand, performance, effort, and frustration. All six studies measure cognitive load in comparative experimental settings. We identify mixed findings regarding LLM-assistants’ impact on mental cognitive load. In fact, a set of studies reports improvements [PS13, PS23, PS38], others neutral effects [PS2, PS8], and only one study reports a significantly worse experience in terms of frustration level [PS12]. For instance, [PS38] reports that Copilot reduces both perceived effort and mental demand for novice programmers. Similarly, [PS23] develops a custom questionnaire to assess cognitive load during programming exam tasks. Their findings show that students using Google Bard report lower mental effort compared to those relying on conventional search engines.

In contrast, [PS12] observes no significant difference in overall cognitive load between students using ChatGPT (GPT-4) and those using a traditional web browser, but does report a statistically significant increase in frustration for the ChatGPT group. Similarly, [PS2] finds that participants rate LLM-assisted tasks as equally demanding and effortful as tasks completed without LLM-assistants. Lastly, [PS8] finds no statistically significant differences across all NASA-TLX dimensions when comparing coding with and without ChatGPT (GPT-3.5).

The variability in reported effects of LLM-assistants on cognitive load highlights the complexity of evaluating mental effort in software development settings. These differences likely stem from diverse operationalizations of cognitive load, differences in participants’ expertise, task design, and the capabilities of LLM-assistants across

studies. This highlights the need for more standardized methodologies and multi-modal assessment strategies to draw robust conclusions about the cognitive impact of LLM-assistants.

**5.3.4 Econometric analysis.** Productivity is a concept primarily inherited from economics and project management. Two complementary studies investigate the impact of LLM-assistants using quantitative econometric analysis of productivity metrics [PS26, PS32] (see Table 7). The first study [PS32] leverages Time-Cost-Quality (TCQ) conceptual framework to conduct a comparative survey of over 1,000 large firms from 2021 to 2023, examining the effect of GenAI on labor productivity across different domains, including coding and content production.

Productivity is assessed in terms of both throughput (i.e., time efficiency) and quality (i.e., correctness of output). The study finds that coding exhibits the highest reported gains, with an average 24% improvement in throughput and 26% in quality. A complementary study by the same author [PS26] investigates the use of LLM-based pair programming through a survey of 70 large global companies. While the findings confirm that LLM-assistants can enhance development throughput, the study also identifies a critical trade-off: increased throughput is negatively correlated with code quality ( $r = -0.45$ ). The study suggests that while LLM-assistants can enhance productivity, their effectiveness depends heavily on organizational readiness and the ability to balance speed with software quality.

#### RQ1: Summary

Among all primary studies, laboratory experiments are the most common strategy (38%, 15 out of 39). Mixed-methods designs are prevalent (69%, 27 out of 39) among empirical methods, often combining user experiments with surveys. Time to completion is the most frequently used performance metric (31%, 12 out of 39). Acceptance rate is a frequently used behavioral metric, though some studies caution against its overuse. Cognitive load findings are mixed: 6 studies use NASA-TLX, but results vary from reduced effort to increased frustration.

#### 6 RQ2: What is the impact of LLM-assistants on software developer productivity?

We conduct a thematic analysis of the findings reported in each primary study. Our analysis reveals several recurring themes, with multiple studies identifying common benefits and risks associated with the use of LLM-assistants. Figure 6 summarizes the frequency of discovered themes, where each theme is reported as a benefit or risk across the primary studies.

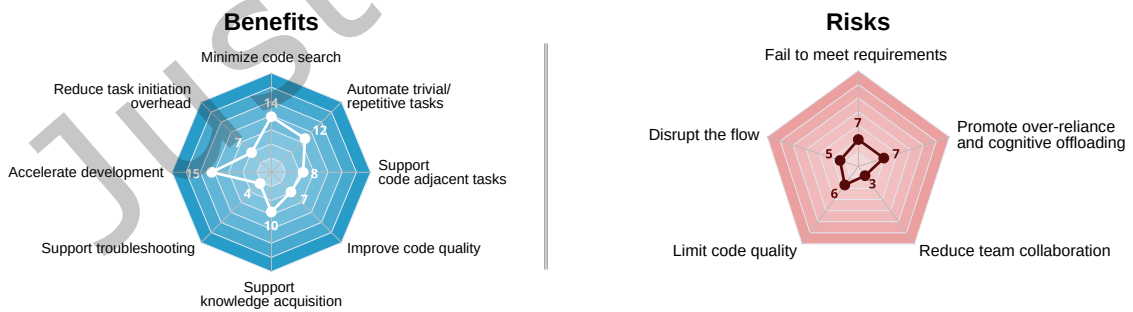


Fig. 6. Radar plots summarizing how frequently each theme appears as benefit (left) and risk (right) across primary studies on LLM-assisted development.

## 6.1 Benefits

Table 8. Summary of LLM-assistants benefits on developer productivity

| Theme                             | Summary                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Accelerate software development   | Studies highlight through self-reported methods that LLM-assistants accelerate software development [PS5, PS11, PS19, PS21, PS22, PS30, PS33, PS34, PS35], and quantitative measures demonstrate that LLM-assistants can reduce task completion time [PS4, PS8, PS13, PS25, PS28, PS29].                                                                                               |
| Minimize online code search       | Study participants in [PS5, PS9, PS11, PS14, PS22, PS33, PS36] noted that LLM-assistants reduce the effort of traditional online search, with developers preferring them over Stack Overflow and search engines [PS15, PS34]. Controlled experiments also report benefits over online search [PS8, PS13, PS24] with some studies having mixed results [PS1, PS18].                     |
| Automate trivial/repetitive tasks | LLM-assistants help minimize repetitive coding [PS7, PS9, PS11, PS31] by generating boilerplate code [PS2, PS5, PS11, PS19, PS27, PS33, PS34] and reducing keystrokes and typing effort [PS11, PS14]. Test generation and CI/CD automation are key use cases [PS29].                                                                                                                   |
| Support knowledge acquisition     | Studies find LLM-assistants helpful in learning and knowledge acquisition as a direct [PS7, PS9, PS11, PS15, PS21, PS31, PS34, PS36] and indirect benefit [PS2, PS6]. LLM-assistants are commonly used as an expert consult, with 75% of respondents finding them helpful for learning [PS7] and lowering the entry barrier to a new frameworks [PS15, PS34].                          |
| Support code-adjacent tasks       | LLM-assistants are found helpful in the ideation process [PS36], requirements specifications [PS28, PS36], documentation [PS5, PS9, PS11, PS36], and quality assurance [PS11, PS24]. Developers also use LLM tools for emails, meeting minutes, onboarding documentation, and documenting issues [PS4, PS34].                                                                          |
| Reduce task initiation overhead   | Participants report benefits at the early stages of projects [PS2, PS33] and highlight LLM-assistants' ability to reduce the entry barrier [PS24]. LLM-assistants also support building proof-of-concept applications [PS11]. Developers utilize these tools to generate initial code scaffolding [PS8], planning and initial structuring of ideas [PS21, PS34].                       |
| Improve code quality              | LLM-assistants have the ability to enhance the quality of code [PS33], which is seen as a key advantage [PS9]. Studies find improvement in code quality [PS18, PS21], with metrics such as cyclomatic complexity, code coverage, technical debt, defect density [PS37], code translation error rate [PS2], code smells [PS27, PS37], defect rate [PS37], and number of defects [PS27]. |
| Support debugging/troubleshooting | Participants leverage LLM-assistants to help interpret error messages [PS36] and suggest potential fixes [PS19] without the need to consult extensive documentation [PS9]. LLM-assistants enable faster bug identification and early defect detection [PS21].                                                                                                                          |

**6.1.1 Accelerate software development.** Time remains a central consideration in many definitions of developer productivity [29, 59, 60, 61] as time is both a valuable and constrained resource within development workflows.

Participants from several empirical studies, particularly those using self-reported methods, suggest that LLM-assistants can accelerate software development [PS11, PS19, PS21, PS30, PS33, PS34, PS35]. Participants often report that LLM-assistants help maintain a state of flow (e.g., “stay in the flow”) [PS11, PS22, PS33] and contribute to a perceived increase in productivity [PS30, PS33]. Supporting this perception, a qualitative analysis of open-ended feedback on an LLM-based code completion tool [PS5] finds “accelerate coding” to be the second most frequent theme, mentioned in 14 responses (20%). Additionally, a 10-week field study with 90 developers at a large firm reports that 52% of respondents perceived productivity boosts from using GitHub Copilot, with ratings progressively increasing week-over-week [PS30].

Complementing these self-reports, studies using quantitative measures demonstrate that LLM-assistants can reduce task completion time [PS8, PS13, PS25, PS28, PS29]. For instance, a case study on the integration of LLMs in the software development lifecycle (SDLC) of a pension plan website [PS28] finds that the required effort decreased from 75 person-days to 22 person-days, representing a productivity gain of 71%. Similarly, statistical significance in time completion has been observed in coding puzzles. This aligns with one of the modes of human–AI interaction described by Barke, James, and Polikarpova [62] as the acceleration mode. Controlled experiments also report efficiency gains ranging from 21% to 45% depending on task type [PS29]. An analysis of 608 GitHub project teams, comparing matched human-only and human-bot teams based on repository activity metrics, finds that human-bot teams showed significantly higher productivity across all team sizes [PS4].

**6.1.2 Minimize online code search.** Minimizing online code search is the most frequently discovered finding that highlights LLMs benefit (see Figure 6). Indeed, multiple studies highlight the potential of LLM-assistants to reduce the effort required for information retrieval [PS5, PS9, PS11, PS14, PS22, PS33, PS36], with developers often preferring LLM-assistants over searching for solutions via traditional online resources, including Q&A platforms such as Stack Overflow [PS9, PS11, PS14, PS15, PS33, PS34] and search engines like Google and Bing [PS9, PS15].

Reducing the need for online search offers several benefits, including helping developers maintain a state of flow (e.g., “stay in the flow”) [PS22], improving the speed of syntax recall [PS11], facilitating the discovery of unfamiliar APIs [PS5, PS11], and providing an alternative in situations where online search methods fail to deliver [PS9]. LLM-assistants are perceived as a more productive alternative to conventional code search, even when additional effort is required for validation [PS36].

Several controlled experiments examine this shift by directly comparing LLM-assisted workflows with traditional online search [PS1, PS13, PS24]. In a user study involving 24 participants, [PS24] employs the *SPACE* framework to compare three conditions: traditional web search, code completion (i.e., Copilot), and interaction with a conversational agent (i.e., ChatGPT), finding significant productivity gains for both code completion and conversational agents across all five *SPACE* dimensions—satisfaction, performance, activity, communication, and efficiency. Similarly, [PS13] shows that using an LLM-assistant plugin for code comprehension leads to statistically significant improvements in task completion rates compared to conventional web search. However, [PS1] reports no statistically significant differences in task completion time or correctness when introducing a PyCharm plugin designed to reduce reliance on Stack Overflow, suggesting that benefits vary across tools or tasks.

This task-dependence is further supported by comparative evidence. [PS18] conducts a study with 44 participants comparing ChatGPT and Stack Overflow across algorithmic problems, library usage, and debugging tasks, finding higher-quality outputs for ChatGPT in algorithmic and library tasks, while Stack Overflow performs better for debugging, with no statistically significant difference in task completion time.

Qualitative interviews contextualize these findings by investigating how developers interpret this trade-off in practice. Several developers report transitioning from Stack Overflow to ChatGPT as a primary information source due to faster, more tailored responses [PS34].

**6.1.3 Automate trivial/ repetitive tasks.** Using LLM-assistants help minimize repetitive coding [PS7, PS9, PS11, PS31] and reduces trivial tasks by generating boilerplate code [PS2, PS5, PS11, PS19, PS27, PS33, PS34]. More specifically, some studies report a reduction in keystrokes and typing effort [PS11, PS14]. Test generation emerges as a key automation use case, with developers viewing unit tests as repetitive tasks well-suited to LLM assistance [PS29]. The broader impact of offloading cognitively repetitive work is further highlighted through a Delphi judgment study conducted with 14 industry professionals to discuss the future of SE in the age of LLM-assistants [PS31]. The study anticipates that LLM-assistants could be used to automate all routine tasks, hence freeing up developer time for more complex tasks.

**6.1.4 Support knowledge acquisition.** The benefits of LLM-assistants extend beyond artifact generation (e.g., source code, test cases). Professional developers increasingly perceive these tools as a valuable aid for learning and knowledge acquisition [PS11, PS21, PS36]. LLM-assistants are found to support knowledge acquisition both as a direct [PS7, PS9, PS11, PS15, PS31, PS34, PS36] and indirect benefit [PS2, PS6]. The authors of [PS2] highlight the concept of knowledge acquisition as an indirect benefit of using LLM-assistants. Although the main goal is to speed up the development process, 69% of participants report that the employed code translation LLM-assistant enhanced their learning (e.g., taught them new aspects of Python) [PS2].

Further evidence is provided by Khojah et al. [PS7], who analyze developer interactions with ChatGPT. Their findings show that expert consultation is the most common use case, accounting for 62% of the analyzed conversations [PS7]. Participants from the same study further highlight such benefits, where 75% of survey respondents report ChatGPT as a helpful learning tool [PS7]. LLM-assistants are also found to lower the barrier of learning new frameworks, enabling developers to start immediately rather than navigating extensive tutorials [PS15, PS34]. Additionally, one study involving 14 experts in SE finds that enhancing learning and teaching is the most probable future scenario with LLM-assistants [PS31].

**6.1.5 Support code-adjacent tasks.** The benefits of LLM-assistants extend beyond coding tasks to support code-related activities. For instance, studies highlight the use of LLM-assistants for ideation [PS36] by exploring different solutions options, requirements specifications [PS28, PS36] including functional and non-functional requirements, documentation [PS5, PS9, PS11, PS36] such as in-code documentation and API documentation, and quality assurance [PS11, PS24]. Developers also use LLM tools for composing emails, generating meeting minutes, and creating onboarding documentation [PS34]. AI assisted teams document significantly more GitHub issues and coordinate more effectively through improved information externalization [PS4].

**6.1.6 Reduce task initiation overhead.** A common reported benefit across primary studies is the use of LLM-assistants to support task or project initiation [PS2, PS8, PS11, PS21, PS24, PS33, PS34]. Several studies note that developers rely on these tools as a *starting point* for a project or task [PS2, PS33], effectively lowering the entry barrier [PS24]. These tools help developers build momentum by reducing the time and cognitive effort required during the early stages of a project. For example, developers highlight how LLM-assistants can accelerate the development of proof-of-concept applications by generating multiple candidate implementations for the same task [PS11] generating an initial structure when starting new topics [PS21].

In a qualitative controlled experiment, [PS8] analyzes the interactions of the developers with ChatGPT and finds that 55% of participants (17 out of 31) used the assistant primarily to generate initial code scaffolding. After this initial phase, participants transitioned to more independent workflows by refining and correcting the code themselves and only relying on the LLM-assistant for targeted questions.

**6.1.7 Improve code quality.** Studies highlight the ability of LLM-assistants to improve code quality [PS2, PS9, PS21, PS27, PS33, PS37]. Interview participants in [PS33] report using these tools to rewrite and improve the quality of the code. Similarly, 14% of survey respondents in [PS9] identify improved code quality as a key advantage of LLM-assistants.

These self-reported perceptions are further supported by empirical evidence. [PS37] compares ten projects developed with the support of LLM-assistants and ten developed without such assistance. The authors evaluate six code quality metrics, including cyclomatic complexity, code coverage, code smells, technical debt, and defect density. They report an 18% improvement across all metrics for projects developed with LLM-assistants. Similarly, [PS27] conducts a case study involving five development teams to examine changes in code quality before and after adopting Copilot. Their findings indicate that three of the five teams experience a measurable reduction in code smells, and all five teams show a decrease in the number of software defects following the integration of LLM-assistants (i.e., Copilot), into their development workflows.

Two controlled experiments further highlight these code quality improvements. In the controlled study [PS18], the authors compare the code produced by participants using ChatGPT (i.e., treatment group) with the one produced by those using Stack Overflow (i.e., control group). The results show higher code quality for the ChatGPT group in algorithmic and library usage tasks, although the control group outperformed in debugging tasks. Similarly, [PS2] evaluates code translation quality with and without LLM-assistant (i.e., TransCoder). The authors measure error rates using several translation-related metrics (e.g., Translation Error, Language Error, Spurious Error) and find that the group supported with LLM-assistants exhibits fewer translation errors compared to the control group, resulting in a 51% reduction in error rate.

**6.1.8 Support troubleshooting / debugging.** As illustrated in Figure 6 and Table 8, recent studies show that LLM-assistants now play an active role in supporting developers during debugging and troubleshooting [PS36]. These tools help interpret error messages by explaining potential causes and suggesting actionable fixes [PS19]. Additionally, several developers report that LLM-assistants accelerate the debugging process, by enabling faster bug identification and early defect detection by recognizing patterns and errors that developers may miss at manual check [PS21], and eliminating the need to consult extensive documentation [PS9].

## 6.2 Risks

Table 9 shows several risk themes identified across the primary studies that may affect developer productivity. In this section, we describe the five risks categories: limit code quality, fail to meet requirements, promote over-reliance and cognitive offloading, reduce team collaboration, and disrupt the flow.

**6.2.1 Fail to meet requirements.** Developers acknowledge that not all the suggestions of LLM-assistants are accurate [PS33]. Many survey participants in [PS11, PS14] mention that LLM-assistants often fail to meet both functional and non-functional requirements. Some developers perceive them as difficult to control [PS14], noting instances where responses are out of context [PS14, PS19] or tend to over-deliver by providing too much information or repetitive code [PS14, PS24]. For example, 50% of participants report missing or misunderstanding the requirement context as the two main issues encountered with ChatGPT 3.5 [PS19]. Generic or inaccurate code suggestions often require additional effort to modify and refine, leading to an iterative process of prompt refinement and learning how to interact effectively with LLM-assistants [PS35]. Benchmarking studies quantify these limitations, with code generation correctness scores ranging from 31.1% (Amazon CodeWhisperer) to 65.2% (ChatGPT) [PS29].

**6.2.2 Promote over-reliance and cognitive offloading.** A heavy reliance and excessive trust in LLM-assistants raises concerns about the erosion of critical thinking skills, especially for novice developers and students [PS12, PS20, PS39]. For instance, authors of [PS39] develop an AI tutor that limits direct interaction with ChatGPT through predefined prompts, aiming to promote critical thinking and reduce dependence on LLM-assistants for every minor challenge. However, students still expressed concerns post-experiment, noting that reliance on the LLM tutor might hinder their learning progress [PS39]. The authors acknowledge this issue and highlight the need for further refinement of the tool.

Table 9. Summary of reported risks associated with LLM-assistants in developer productivity

| Theme                                          | Summary                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fail to meet requirements                      | LLM-assistants often do not meet functional or non-functional requirements [PS11, PS14]. Developers suggest that not all LLM-assistants' outputs have good accuracy [PS33, PS35]. This is due to the limited controllability of LLMs' output [PS14], as they can be out of context [PS14, PS19] and tend to over-deliver by providing too much information or repetitive code [PS14, PS24]. Code generation correctness ranges from 31–65% [PS29].                                 |
| Promote over-reliance and cognitive offloading | Several studies raise concerns on the diminishing of critical thinking skills among novices and students [PS12, PS20, PS39]. In professional settings, instances of automation complacency have also been reported [PS23]. Developers worry about skill erosion and loss of creativity [PS15]. To address these issues, studies recommend promoting cautious and informed LLM-assistants use, emphasizing interactive engagement over passive acceptance [PS8, PS20, PS34].        |
| Limit code quality                             | Concerns were raised about the quality and accuracy of LLM-generated codes [PS9, PS22]. Vulnerabilities and bugs can be introduced if a developer overestimates the capabilities of the tool [PS19]. Working with LLM-assistants might not yield better code quality [PS8, PS26].                                                                                                                                                                                                  |
| Disrupt the flow                               | LLM-assistants can disrupt developers' flow with unwanted suggestions [PS33], interface switching [PS24], verbose answers [PS24], and inadequate speed of code suggestions [PS24]. The simultaneous use of multiple code completion tools can disrupt developer flow, particularly when competing suggestions are presented [PS5]. Developers spend an average of 51.5% of coding time in LLM interaction states [PS22]. Human-AI teams may experience notification fatigue [PS4]. |
| Reduce team collaboration                      | Relying on LLM-assistants introduces the risk of hindering team collaboration and communication [PS7]. Traditional help channels have become less active as developers prefer AI assistance [PS15]. Teams report losing organic conversations and synergy [PS15]. This highlights the need to investigate the impact of LLM-assistance for both human-human and human-agent collaboration and communication [PS3].                                                                 |

Over-reliance and automation complacency have also been documented among professional software engineers. In a study involving a programming exam with Google Bard, [PS23] observes all three characteristics of automation complacency, as described by Parasuraman and Manzey [63]: human monitoring of an automated system, infrequent monitoring, and degraded performance. Concerns about skill erosion extend across experience levels. Survey respondents express concerns about overreliance, reporting diminished ability to think independently [PS15].

These findings highlight the need to promote responsible LLM usage. Several studies advocate for more interactive and reflective engagement with LLM outputs [PS8], cautioning against blind trust in automated responses [PS20] and encouraging users to understand the tools' capabilities and limitations. Finding the right balance between leveraging AI support and maintaining developer competence remains an open challenge [PS34].

**6.2.3 Disrupt the flow.** Studies find that LLM-assistants can disrupt developer flow [PS5, PS33]. Issues that impact developer state of flow have been attributed to various kinds of interruptions, including unwanted LLM

suggestions [PS33], interface switching, and verbose answers [PS24]. For example, in a laboratory experiment [PS24], some professional developers find Copilot distracting, as the speed of code suggestions does not allow sufficient time for code understanding. Distraction also occurs when LLM-assistants work in tandem and compete to display suggestions [PS5].

Authors of [PS22] investigate developers' interaction with code recommendation systems and their impact on flow by modeling user behavior while using tools such as Copilot. Findings show that developers spend an average of 51.5% of their coding session time in LLM interaction states, such as verifying suggestions, prompt crafting, and deferring thought. In open source contexts, human-bot teams may experience notification fatigue from increased automated activity [PS4]. These findings highlight the temporal and cognitive costs that these tools may introduce.

#### 6.2.4 *Limit code quality.*

Multiple studies raise concerns about the quality of generated code. For instance, [PS9] reports that 13% of survey respondents raise concerns about the quality and accuracy of LLM-generated code. Similar concerns are raised by 29% of survey participants when using Copilot [PS22]. Code quality issues arise when developers overestimate the capabilities of such tools, which can introduce vulnerabilities and bugs. LLM-assistants often struggle with optimization and refactoring tasks, especially when lacking semantic context. Moreover, in [PS19], 38% of the surveyed participants point to erroneous code as one of the limitations of ChatGPT, while 29% report limitations due to inefficient code. User experiments conducted by the authors of [PS8] reveal no significant improvements in code quality or correctness with LLM-assistants, with a slightly worse quality average for the ChatGPT group.

When analyzing the relationship between reported productivity and code quality gains in the context of LLM-assistant, a large industry study involving 70 large global companies [PS26] reports a moderate negative correlation ( $r = -0.45$ ) between the two. These findings highlight that increased productivity through the support of LLM-assistants does not necessarily lead to improvements in code quality. Hallucinations remain a persistent concern [PS34]. LLM-assistants may produce non-functional solutions or outputs containing deprecated libraries that fail review [PS34]. The lack of contextual understanding extends to organizational and domain-specific knowledge. LLM-assistants struggle to account for company-specific coding standards, architectural conventions, or project constraints, potentially generating code that violates established conventions of the surrounding codebase [PS34].

Code quality is the one theme that has been reported as both a benefit and a risk (see Figure 6, Table 8 and 9). The variation in findings across studies can be attributed to differences in both experimental scope and evaluation methods. Some studies examine small, isolated programming tasks, while others assess complex, real-world systems. Moreover, researchers use diverse metrics (e.g., such as cyclomatic complexity, defect density, and code coverage) that capture different dimensions of code quality. These contextual and methodological differences make direct comparison difficult and help explain why LLM-assisted code quality appears both improved and degraded across the literature.

#### 6.2.5 *Reduce team collaboration.*

Relying on LLM-assistants can negatively impact productivity by reducing team collaboration and communication [PS7]. For instance, a field study [PS7] observes that excessive use of LLM-assistants may lead developers to favor consulting a chatbot over a colleague. In fact, the overconfidence of LLM-assistants' responses can create the impression that team discussions are unnecessary, reducing opportunities for communicative learning and discovery [PS7]. The intrinsic nature of LLM-assistants also plays a role in reduced team collaborations. For instance, current conversational LLM-assistants offer limited support for team collaboration, as they primarily support one-on-one interactions and are not well-suited to facilitating effective team coordination. Traditional help channels have become less active, with developers preferring AI assistance, in part to avoid exposing knowledge gaps to colleagues [PS15]. Teams report losing the organic conversations and

synergy that come from bouncing ideas off each other [PS15]. These findings highlight the need for future studies to further investigate how LLM-assistants affect human-human collaboration and how they can be designed to foster team collaboration [PS3].

## RQ 2 - Summary

Studies report mixed findings on the use of LLM-assistants, revealing both notable benefits and critical risks. The most frequently reported benefits include accelerated development, minimizing code search, and automating trivial or repetitive tasks. At the same time, primary studies identify several risks, such as failing to meet requirements, promoting over-reliance and cognitive offloading, and disrupting developer flow. Code quality emerges as a particularly contested area, with evidence pointing to both improvements and degradations depending on context. These discrepancies underscore the need for further investigation and the development of strategies to ensure that code quality is maintained when integrating LLM-assistants into software development workflows.

## 7 RQ3: Which dimensions of developer productivity are investigated, and how do these dimensions map onto the SPACE framework?

The diverse methodologies and wide variation in reported findings across primary studies (see Section 5 and Section 6) highlight the need for a structured lens to interpret how productivity is conceptualized in the literature, especially in the context of rapidly evolving development with LLM-assistants. However, existing approaches to measuring productivity often fall short of capturing this diversity. Traditional metrics such as Lines of Code (LOC) and Function Points, have long been criticized for offering only a narrow view of productivity centered on output quantity. Likewise, quantitative performance frameworks such as the DevOps Research and Assessment (DORA) model [64], though valuable for assessing delivery speed and deployment frequency, do not reflect the human, social, and cognitive dimensions that shape productivity in Human-AI Interaction. Recognizing these limitations, we adopt a multidimensional perspective grounded in the *SPACE* framework, which integrates objective outcomes (e.g., performance, activity, efficiency) with human-centered constructs (e.g., satisfaction and collaboration).

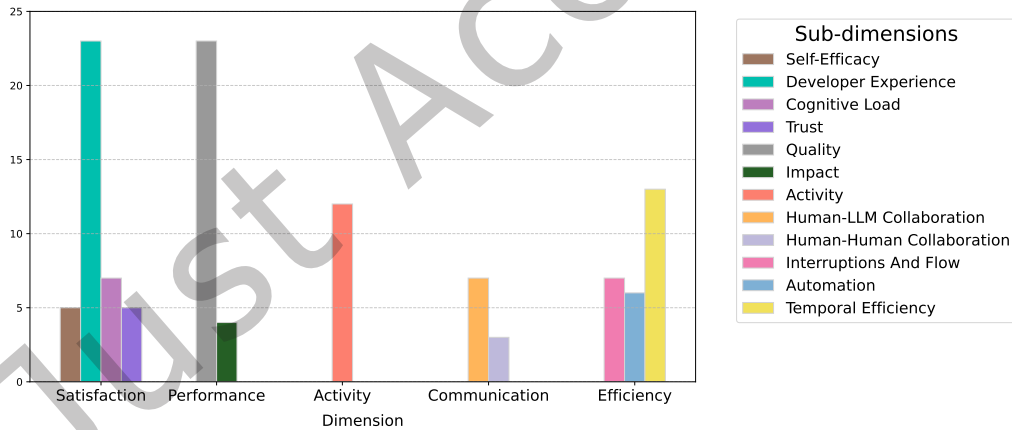
Therefore, we leverage the *SPACE* framework [19], a widely used framework developed by Microsoft researchers, as an organizing model to understand productivity from a practical perspective. We choose the *SPACE* framework because it defines productivity as a multi-dimensional construct that covers a diverse range of study instrumentation from measurement metrics (e.g., code quality) to perceptions (e.g., satisfaction). The *SPACE* framework also reflects a modern development workflow, aligned with the complexities of LLM-assisted development, which emphasizes dimensions like collaboration and trust. Its extensible and adaptable nature allows for contextual adaptation, as it does not prescribe fixed metrics but offers a conceptual structure that can be tailored to specific empirical contexts. We map each primary study to the five dimensions of the *SPACE* framework: Satisfaction, Performance, Activity, Communication & collaboration, and Efficiency (see Figure 7).

To synthesize and compare findings from our primary studies, we leverage thematic analysis, employing an adapted version of the *SPACE* framework. We use the five dimensions of the *SPACE* framework. To provide more granularity, we further refine these dimensions by including sub-dimensions. Specifically, we adapt some sub-dimensions from relevant related work [65]. When a specific concept was not captured by an existing sub-dimension, we ensure comprehensive coverage by including emerging sub-dimensions derived from our data. A structured taxonomy for our analysis is provided in Table 10. Figure 7 illustrates the distribution of the sub-dimensions across each dimension of the *SPACE* framework.

Our findings reveal that the majority of studies (90%, 35 out of 39) adopt a multidimensional perspective on productivity, with only four studies taking a uni-dimensional perspective. This indicates a shift away from

Table 10. Mapping of primary studies to SPACE dimensions and derived sub-dimensions. With sources for each sub-dimension are cited from prior work.

| Dimension     | Sub-dimensions                  | Primary Studies                                                                                                                        | %   |
|---------------|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----|
| Satisfaction  | Developer experience [46]       | [PS1, PS5, PS7, PS9, PS11, PS13, PS14, PS16, PS17, PS18, PS19, PS20, PS21, PS22, PS24, PS25, PS27, PS30, PS34, PS35, PS36, PS37, PS39] | 77% |
|               | Self-efficacy [19, 65]          | [PS3, PS12, PS21, PS33, PS38]                                                                                                          |     |
|               | Trust [65]                      | [PS3, PS7, PS12, PS19, PS23]                                                                                                           |     |
|               | Cognitive load                  | [PS2, PS8, PS12, PS13, PS23, PS25, PS38]                                                                                               |     |
| Performance   | Quality [19, 65]                | [PS1, PS2, PS3, PS5, PS8, PS10, PS12, PS15, PS16, PS18, PS19, PS20, PS22, PS23, PS24, PS25, PS26, PS27, PS28, PS29, PS32, PS37, PS38]  | 64% |
|               | Impact [19]                     | [PS26, PS31, PS32, PS34]                                                                                                               |     |
| Activity      |                                 | [PS4, PS5, PS6, PS10, PS12, PS15, PS16, PS17, PS20, PS22, PS24, PS27]                                                                  | 31% |
| Communication | Human-LLM collaboration         | [PS13, PS16, PS22, PS24, PS34, PS36, PS39]                                                                                             | 26% |
|               | Human-human collaboration       | [PS3, PS7, PS27]                                                                                                                       |     |
| Efficiency    | Temporal efficiency [19]        | [PS1, PS6, PS8, PS13, PS15, PS18, PS23, PS24, PS26, PS27, PS28, PS29, PS37]                                                            | 59% |
|               | Interruptions and flow [19, 65] | [PS3, PS11, PS16, PS22, PS31, PS33, PS35]                                                                                              |     |
|               | Automation                      | [PS9, PS11, PS17, PS22, PS29, PS34]                                                                                                    |     |

Fig. 7. Distribution of sub-dimensions across each dimension of the *SPACE* framework.

singular-dimension perspectives toward a more holistic understanding of the complex nature of productivity in software engineering (See figure 8). However, only 44% of the studies (17 out of 39) examine three or more of the *SPACE* dimensions, with just 15% (6 out of 39) addressing four or more dimensions. This highlights the need for future work to capture the full breadth of how LLM-assistants impact productivity. We find that the most

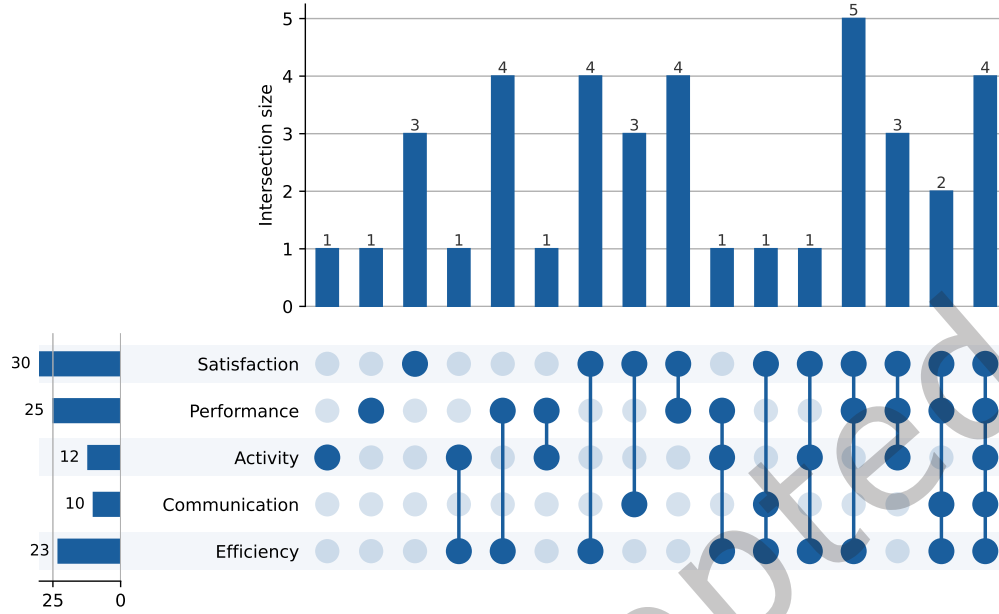


Fig. 8. The distribution of investigated SPACE dimensions and their overlap.

co-occurring combinations involve Satisfaction, Performance, and Efficiency. Out of the list of primary studies, the most frequent combination is Satisfaction-Performance-Efficiency (5 out of 39).

Table 10 and Figure 8 show that **Satisfaction** is the most studied dimension, addressed by 77% (30 out of 39) of primary studies. This dimension captures developers' feelings about their work with LLM-assistants, which is mainly captured through self-reported instruments. Our analysis of this dimension reveals five fine-grained sub-dimensions (i.e., *developer experience*, *self-efficacy*, *trust*, and *cognitive load*). Most studies within the satisfaction dimension focus on the concept of *developer experience*, which encompasses developers' perceptions, feelings, and values regarding their interactions with LLM-assistants [46], as well as the perceived importance of LLM-assistants and their ease of use (e.g., developers' feedback or the Technology Acceptance Model (TAM)). *Cognitive load* is the second most commonly investigated satisfaction sub-dimension, assessed using instruments such as NASA-TLX or custom surveys. *Self-efficacy*, the belief in one's ability to complete tasks, is explored using both validated and custom-designed tools. Finally, We find that *well-being* is not examined by any of the empirical studies. This aligns with the observations of [66], who highlight that developers' well-being and mental health are frequently overlooked in software engineering research.

**Performance** is the second most studied dimension covered by 64% (25 out of 39) of primary studies. Performance mainly concerns the final outcomes of software development activities. The majority of studies investigate the *quality* sub-dimension of performance using instruments such as passing unit tests, functional correctness, and code smells (see Table 11). The *impact* sub-dimension is addressed in only three studies and mainly investigates how LLM-assistants impact final product outcomes [19]. This sub-dimension focuses on business-related metrics, including cost savings, product quality improvements, and delivery speed (see section 5.3.4).

**Efficiency** is the third most studied dimension covered by 59% (23 out of 39) of primary studies. This dimension reflects the capacity to complete tasks efficiently with minimal interruptions or time delays, as it aims to minimize unnecessary delays and optimize the flow of task handoffs [19]. We highlight three sub-dimensions explored by

primary studies (i.e., *temporal efficiency, automation, interruptions and flow*). Efficiency is often investigated from the temporal perspective, measured using task completion metrics or via developer perceptions. *Automation* is another important angle of efficiency, with studies reporting how LLM-assistants are used to offload repetitive tasks such as writing boilerplate code [PS9, PS11]. Finally, studies examine efficiency in terms of *interruptions and flow*, with some highlighting reduced cognitive interruptions and others noting new forms of distraction introduced by the LLM-assistant.

The **Activity** is one of the least explored dimensions (31%, 12 out of 39). Activity is often paired with efficiency and performance and focuses on counts and frequency measures while performing a given task [19]. Studies included in our review often measure activity as the count of actions or tasks developers take during their interactions with LLMs (e.g., acceptance rate, number of tasks completed)[PS5, PS6, PS16, PS24]. For example, Ziegler et al. [PS16] measures the dimension of activity at a finer granularity, regarding the count of actions developers take during their interactions with Copilot (e.g., acceptance rate, suggestions shown rate, completions changed or unchanged).

**Communication** is the least investigated dimension across primary studies (26%, 10 out of 39). Communication focuses on how developers and teams communicate and share knowledge [19]. The majority of the studies mapped to the communication dimension investigate the dimension in terms of *human-LLM collaboration* (7 out of 10) , which includes analysis of interaction patterns between participants and LLM-assistants. While only three (3 out of 10) examine *human-human collaboration* with LLM in the loop. This shows a gap in our understanding of how LLM-assistants influence team communication or coordination, which is also highlighted by prior studies [66, 67]. Given that emerging concerns regarding reduced team collaboration due to over-reliance on LLM-assistants (see Section 6.2.5), future studies should incorporate more investigation on team dynamics to better understand their impact in the LLM-assisted workflows.

Table 11. Quality metrics by study.

| Metric                              | Primary Studies              |
|-------------------------------------|------------------------------|
| Passing Unit Tests                  | [PS2, PS8, PS18, PS19, PS38] |
| Functional Correctness and Accuracy | [PS3, PS10, PS18, PS28]      |
| Code Smells                         | [PS12, PS27, PS37]           |
| BLEU Score                          | [PS5, PS10]                  |
| Halstead Complexity Measures        | [PS24, PS28]                 |
| Cyclomatic Complexity               | [PS1, PS37]                  |
| Translation Error Rate              | [PS2]                        |
| Maintainability Index               | [PS24]                       |
| Cognitive Complexity                | [PS27]                       |
| Defect Density                      | [PS37]                       |
| Defect Rate                         | [PS27]                       |
| Technical Debt                      | [PS37]                       |
| Code Coverage                       | [PS37]                       |

**RQ3 - Summary**

Our analysis, framed by the *SPACE* framework, reveals that the majority of studies (90%, 35 out of 39) adopt a multidimensional view of productivity in SE, mostly combining two or more *SPACE* dimensions. However, only 15% (6 out of 39) of the studies examine four or more *SPACE* dimensions. Satisfaction (77%, 30 out of 39) is the most frequently investigated, followed by Performance (64%, 25 out of 39) and Efficiency (59%, 23 out of 39). In contrast, Activity (31%, 12 out of 39) and Communication (26%, 10 out of 39) is the least explored dimensions.

**8 Discussion**

This discussion synthesizes findings from 39 primary studies to derive cross-cutting insights, actionable guidance, and research directions on the use of LLM-assistants in software development. We first present an in-depth synthesis of the findings using McLuhan’s Tetrad [68] to explain how LLM-assistants reshape development practices beyond measurable productivity gains. We then distill key takeaways and lessons learned for practitioners, followed by actionable recommendations at the individual, team, and organizational levels. Finally, we identify open issues and research gaps and discuss their practical and ethical implications for the software engineering community.

**8.1 In-depth synthesis across studies using the McLuhan Tetrad**

As established in RQ3 (Section 7), traditional approaches to defining and measuring productivity in software engineering remain limited, even when expanded through multidimensional frameworks such as *SPACE*. While these models clarify how productivity is operationalized, they do not fully capture the deeper transformations that arise when new technologies reshape everyday development practices. The integration of LLM-assistants extends beyond measurable performance gains, since it redefines development practices, decision-making, and team dynamics across the entire software life cycle [PS7, PS27, PS28]. As discussed in Section 6.1, LLM-assistants bring clear productivity benefits, such as improved code quality and accelerated development, yet also introduce risks related to over-reliance, erosion of critical judgment, and reduced collaboration (Section 6.2).

To synthesize these broader transformations, we draw on McLuhan’s tetrad [68], a conceptual framework originally proposed to analyze how emerging media technologies transform human behavior and social organization. The Tetrad complements the *SPACE* framework by shifting the focus from measurement to interpretation. It examines *with what consequences* these socio-technical shifts occur. The framework poses four interrelated questions: 1) *What does the medium enhance?*, 2) *What does it render obsolete?*, 3) *What does it retrieve?*, and 4) *What does it reverse when pushed to the extreme?* Figure 9 illustrates how this lens is applied to analyze the impact of LLM-assistants on software development.

**8.1.1 Enhance.** In McLuhan’s Tetrad, the *Enhance* dimension refers to what technology intensifies in existing practices. In the realm of SE, LLM-assistants have the potential to enhance several aspects of the development process. As summarized in Section 6.1, multiple studies report that LLM-assistants enhance productivity primarily by accelerating development (Section 6.1.1), lowering entry barriers for complex tasks (Section 6.1.6), support knowledge acquisition for both professionals and students (Section 6.1.4), and support debugging and troubleshooting (Section 6.1.8). Taken together, LLM assistants are effective when applied to tasks that align with their strengths, such as boilerplate generation, syntax recall, initial scaffolding, and exploratory prototyping. **Practitioners should leverage LLM-assistants selectively and strategically rather than expecting uniform gains across all development activities.**

**8.1.2 Reverse.** According to McLuhan’s Tetrad, the *Reverse* dimension captures the unintended or counterproductive consequences that arise when technology is pushed to its limits. As discussed in Section 6.2.2, excessive trust

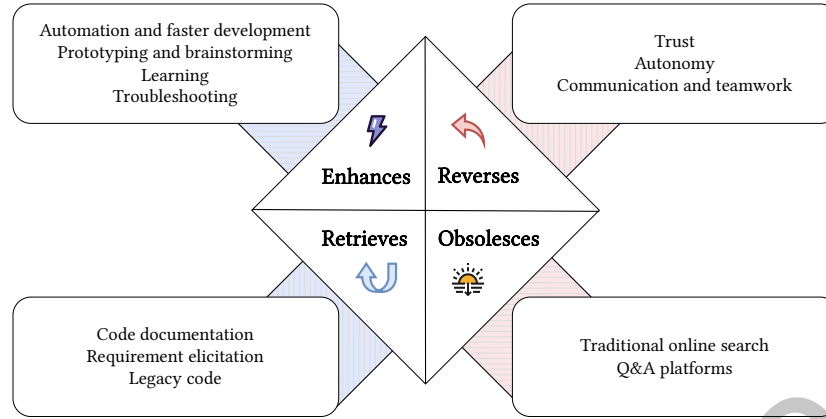


Fig. 9. McLuhan's Tetrad diagram illustrates the implications of LLM assistants on the productivity of software developers. The diagram captures four dimensions. *Enhancement*: how LLM-assistants amplify development speed; *Obsolescence*: which traditional practices are being displaced; *Retrieval*: which previously diminished practices are being revived; and *Reversal*: what adverse effects may emerge when LLM-assistants are pushed to the extreme.

in LLM-assistants can lead to cognitive offloading and automation complacency, while the lack of trust creates frustration and discourages adoption. Over-reliance further contributes to diminished developer autonomy [67] by shifting developers from active code production to reviewing generated output. This reduced reflective engagement can negatively affect code quality (Section 6.2.4) particularly when LLM-generated code is accepted without sufficient validation. Finally, reliance on LLM-assistants may weaken collaboration and peer communication (Section 6.2.5), as developers turn to the LLM-assistant instead of teammates for feedback. Taken together, these findings suggest that productivity gains can diminish when LLM-assistants are used without adequate oversight. Uncritical reliance can undermine reflective practice, collaborative problem-solving, and the development of domain expertise that are essential for long-term software quality. **Practitioners should therefore use LLM-assistants critically: treat generated outputs as preliminary drafts requiring thorough review, maintain active engagement in coding decisions, and balance tool use with collaborative practices to preserve reflective judgment, team communication, and long-term code quality.**

**8.1.3 Obsolesce.** The *Obsolesce* dimension of McLuhan's Tetrad refers to the tools, practices, or roles that are diminished or displaced as a result of adopting new technologies. For LLM-assistants, traditional online search techniques for recalling syntax, or looking up new libraries online (Section 6.1.2). Developers may prefer using conversational agents over searching for solutions or consulting Q&A platforms such as Stack Overflow (Section 6.1.2). While these effects may enhance efficiency, they also risk diminishing the capacity to independently verify information, evaluate competing solutions, and cultivate the search and validation that remain essential when LLM outputs are incorrect or incomplete. **Practitioners should therefore treat LLM-assistants as a complement rather than a replacement for traditional information-seeking. They should continue maintaining proficiency in search and validation practices, cross-check suggestions against reliable sources.**

**8.1.4 Retrieve.** In McLuhan's Tetrad, the *Retrieve* dimension reflects the resurgence or reintegration of practices that had diminished in relevance prior to the adoption of new technology. For LLM-assistants, previously neglected

activities such practice code documentation, which is now being brought back to focus in development workflows with the help of LLM-assistants (Section 6.1.5), and requirements engineering, including requirement elicitation that are traditionally time-consuming due to frequent client communication [69], are gaining momentum through LLM-assistants.

Additionally, LLM-assistants also offer opportunities to revisit legacy systems, a domain often overlooked due to the high cost and complexity and maintaining outdated platforms such as COBOL or Uniface[PS31, PS35]. While support for legacy systems is currently limited[PS14], these tools show promise in expanding the scope of productivity beyond coding speed to include documentation, requirements, and modernization tasks. **Practitioners should leverage LLM-assistants to revive these previously deprioritized practices. Specifically, they can integrate LLM-assisted documentation, requirements elicitation, and legacy system support into development workflows to enhance maintainability, preserve institutional knowledge, and improve long-term software quality, while still combining AI assistance with human expertise.**

**Lessons learned.** Across enhancement, reversal, obsolescence, and retrieval, three overarching lessons emerge:

- (1) Productivity gains are task-contingent, strongest for well-scoped and repetitive activities.
- (2) Uncritical reliance introduces diminishing returns through validation overhead and erosion of reflective practice.
- (3) LLM-assistants reshape, not replace, developer expertise, shifting effort toward evaluation, judgment, and coordination.

## 8.2 Implications and recommendations for software developers

The integration of LLM-assistants into the software development workflow has profound implications, transforming both the individual developer's role and the team's dynamics. While these tools offer benefits, developers must be mindful of the potential risks and adapt their practices to maximize productivity and maintain high-quality code.

**Cultivating and managing trust.** Establishing the appropriate level of trust, presents challenges caused by the trade-off between benefits and risks, as presented in RQ2 (see Section 6). In RQ3 (Section 7), we incorporated trust as a sub-dimension of "Satisfaction", since trust can influence how developers perceive their productivity [PS19]. Excessive trust can lead to automation complacency and over-reliance [PS23], while insufficient trust can cause frustration and underutilization of the tools' benefits. Balancing trust is therefore essential to sustain long-term satisfaction and effective collaboration between developers and LLM-assistants.

Trust can become fragile when these models hallucinate or produce incorrect outputs, such as suggesting non-existent APIs or generating syntactically incorrect code [PS5]. These issues stem from the traditional trade-off between precision and recall in machine learning systems, where models tuned for high recall may increase coverage but at the cost of higher volume of incorrect suggestions [PS5]. This creates a trust barrier between developers and LLM-assistants.

Additionally, LLMs often suffer from a lack of transparency, as they do not provide sources or references for their outputs [PS7] which deepens the trust gap. Implementing AI solutions with transparency mechanisms [PS9] to elucidate their decision-making processes empowers developers to trust and better understand the LLM-assistants' suggestions and decisions. These implications may extend to a much broader scope. Through a questionnaire conducted by [PS19], the authors find a moderate positive correlation between self-reported productivity and trust levels, as participants who report increased productivity with LLM-assistants also exhibit slightly higher levels of trust. However, trust dynamics are not uniform across user groups. For instance, the

study in [PS23] finds that novice developers tend to demonstrate automation complacency, often accepting LLM-assistants suggestions uncritically and with minimal validation. While this may temporarily boost productivity, it also raises concerns about long-term skill development and critical thinking.

**Recommendation 1.** Developers should cultivate calibrated trust by understanding the capabilities and limitations of LLM-assistants. Operationally, this recommends: (i) treating all LLM-generated code as preliminary output requiring validation through testing and review; (ii) cross-referencing suggestions against official documentation for unfamiliar APIs; (iii) acknowledging that LLMs lack project-specific context and may produce hallucinated outputs; and (iv) periodically engaging in unassisted coding to preserve fundamental competencies. Cultivating such informed, critical trust is essential for maximizing benefits while safeguarding code quality, developer autonomy, and long-term skill development.

**Redefining the developer’s role from coder to reviewer.** LLM-assistants often lack awareness of the broader context and intricacies of a complex software project [PS11, PS14]. Developers often mitigate this issue by breaking down the problem [PS11, PS14, PS19] and providing a clear explanation to the LLM-assistants [PS11, PS14]. Consequently, developers increasingly spend time verifying, editing or refining LLM-assistants generated suggestions rather than writing code. One study by Mozannar et al. [PS22] reports that participants spend over 50% of their time in evaluation activities, such as crafting prompts, reviewing suggestions, and editing completion. Similarly, Weisz et al. [PS2] finds that using LLM-assistants for code translation transforms the developer’s responsibility into one of reviewing rather than writing code. Time saved in code generation can be lost in the evaluation and refinement phases, especially for complex tasks. This can lead to diminishing returns on productivity if not managed effectively. [PS26]. These findings align with related work regarding “ironies of automation” [70], where the introduction of automation often shifts users’ roles from production to evaluation. This shift raises practical implications for developers’ evolving roles. New skills such as prompt engineering, critical evaluation of AI-generated code, and contextual judgment are becoming central to day-to-day work. Awareness of automation bias and complacency must also be cultivated to prevent uncritical acceptance of LLM suggestions. Accordingly, developer training should emphasize verification, debugging, reflective use of AI, and collaborative practices that preserve knowledge sharing. At the team level, this role change may alter composition and responsibilities, increasing the value of roles focused on oversight, integration, and code review rather than routine implementation.

**Recommendation 2.** The role of developers is evolving from coder to reviewer when using LLM-assistants. To maximize productivity and code quality, developers must shift their mindset and reallocate time and cognitive resources to tasks like prompt engineering, iterative evaluation, and refining LLM-generated outputs. This includes guiding LLMs with precise context, treating suggestions as drafts requiring human validation, and decomposing complex tasks into smaller, well-scoped sub-problems for more coherent responses. Given that evaluation activities may constitute a considerable amount of time, developers are advised to allocate dedicated time for validation rather than treating it as ancillary, and to adopt systematic approaches for assessing LLM outputs with respect to correctness, security, and adherence to project standards.

**Adapting the development workflow.** The integration of LLM-assistants may disrupt established software development workflow, which can affect team collaboration and individual tasks. At the team level, we describe in section 6.2.5 how LLM-assistants may reduce collaboration among software teams. It remains unclear to what extent the adoption of LLM-assistants leads to a decline in developer knowledge sharing and pair programming. At the individual level, LLM-assistants can disrupt developers’ flow. As described in section 6.2.3, unwanted

or irrelevant LLM-assistants' suggestions can disrupt the developer's flow. For instance, studies report that interruptions from tools like Copilot, especially when suggestions are too frequent or conflict with other tools, can disrupt focus and reduce productivity [PS5, PS24, PS33].

**Recommendation 3.** Developers must adapt their practices and team dynamics to ensure LLM-assistants enhance, rather than hinder, the development workflow. Individually, developers should customize tool settings to control suggestion frequency and limit interruptions. At the team level, preserving collaborative practices like pair programming, code reviews, and architectural discussions is crucial to maintaining communication and knowledge sharing. Teams are encouraged to establish explicit norms governing when to consult colleagues versus LLM-assistants, particularly for decisions affecting shared codebases, and to document LLM-assisted decisions in commit messages or design records to preserve collective awareness.

**Organizational factors and adoption strategy.** The effectiveness of LLM-assistants depends on organizational context. Studies examining productivity at the organizational level (Section 5.3.4) analyze how firm-level or team-level practices shape the adoption and outcomes of LLM-assisted development. Evidence from industry-level econometric analysis shows a moderate negative correlation ( $r = -0.45$ ) between throughput and code quality (Section 6.2.4). These findings highlight the importance of organizational readiness and governance mechanisms that balance speed with quality through clear accountability and quality-assurance processes. Moreover, as shown in 6.2.5, reliance on LLM-assistants can alter communication patterns and reduce collaboration among developers. To sustain long-term benefits, organizations should embed LLM-assistants into workflows through well-defined policies and structured review procedures that ensure AI-generated artifacts receive rigorous oversight. As LLMs continue to advance in capability and autonomy, organizations should periodically revisit their adoption strategies and governance frameworks to ensure that new generations of tools strengthen, rather than disrupt, team collaboration, trust, and code quality.

**Recommendation 4.** Organizations should assess their readiness for LLM-assisted development and adopt strategies that align productivity gains with code quality. This includes investing in training to cultivate calibrated trust, implementing quality-assurance processes for AI-generated artifacts, and fostering a culture that values collaboration and continuous learning alongside automation. Given the documented negative correlation between throughput and quality [PS26], organizations are advised to establish clear policies delineating which tasks are appropriate for LLM assistance, mandate enhanced review processes for LLM-generated code in high-risk modules, and monitor team collaboration patterns to detect and mitigate any erosion of knowledge sharing.

**Professional and Ethical Considerations.** The integration of LLMs into software engineering introduces complex professional and ethical challenges related to accountability, transparency, and fairness. The black-box nature of proprietary LLM systems presents a transparency deficit. These tools frequently do not provide sources or references for their generated code, which hinders developers' ability to conduct rigorous validation and understand the code's lineage or security implications [PS9, PS14]. This lack of transparency contributes to accountability gaps when LLM-generated code introduces defects, vulnerabilities, or potentially copyrighted material.

**Recommendation 5.** Organizations should mandate explicit disclosure of AI-assisted contributions and establish clear accountability frameworks that define ownership, review requirements, and traceability of AI-generated artifacts. Developers must remain vigilant about algorithmic bias, as LLMs trained on skewed internet data can perpetuate unfair or discriminatory patterns. Ethical review and bias testing should therefore be integrated into standard software quality assurance processes, treating AI output as both a technical and socio-technical artifact. Finally, where feasible, practitioners should favor tools and workflows that improve transparency and traceability, enabling informed validation and responsible professional judgment.

### 8.3 Implications and recommendations for researchers

**Open issues and research gaps.** Our synthesis identifies several gaps that require attention from the research community. First, developer well-being is not directly examined by any primary study, despite growing recognition that mental health and sustainable work practices are integral to long-term productivity. Second, human-human collaboration in LLM-mediated workflows remains understudied: only 3 of 10 studies addressing the Communication dimension, leaving unresolved questions regarding how LLM-assistants reshape pair programming, knowledge sharing, and collective code ownership. Third, long-term effects remain largely unknown, as the majority of studies employ short-term laboratory designs that cannot capture cumulative technical debt, sustained versus diminishing productivity gains, or skill erosion over extended periods. Fourth, the field lacks standardized metrics and validated instruments, which impedes cross-study comparison and cumulative knowledge building. Fifth, the conditions under which LLM-assistants improve versus degrade code quality remain unresolved, as this outcome is reported as both a benefit and a risk depending on study context and evaluation criteria.

**Establishing shared practices and cumulative insight.** The current body of research mainly rely on laboratory experiments, which represent the most common research strategy (38%), focusing on controlled tasks to isolate effects. This is critical for early insights. However, the methodological diversity can challenge cross-study comparison and synthesis. For instance, LLM-assistant's effect on code quality is inconsistent, reported as both a benefit and a risk depending on study context and metrics used. Similarly, cognitive load findings are mixed: some studies report reduced mental effort, while others find no effect or increased frustration. These inconsistencies highlight the limits of the current evidence base and motivate context-aware, longitudinal studies that follow developers, teams, and organizations over time. Short-term studies alone cannot determine whether initial gains are sustained or whether they mask longer-term risks such as skill erosion, automation complacency, cumulative technical debt in evolving systems, or whether observed throughput translates into lasting organizational benefits.

**Recommendation 1.** Researchers should adopt shared evaluation frameworks and validated instruments that allow comparability. Future work should include longitudinal, field, and team-based studies that capture long-term outcomes. Specifically, future investigations should: (i) employ standardized cognitive load measures to reconcile the current inconsistency in findings; (ii) track productivity, code quality, and skill development over extended periods rather than single sessions; (iii) conduct organizational case studies that capture real-world complexity; and (iv) triangulate quantitative metrics with qualitative insights to elucidate the mechanisms underlying observed effects.

**On the dimensions of productivity.** Existing work agrees that developer productivity is a multi-dimensional and context-sensitive construct [27]. This complexity is magnified in the context of LLM-assisted development. Our findings provide evidence that the vast majority of studies (90%) examine at least two *SPACE* dimensions. This indicates a positive move from the research community toward a multidimensional perspective. However,

our findings also highlight that only 15% of studies extend beyond three dimensions, indicating that room for improvement remain in terms of providing a more complete evaluation of the productivity concept.

Satisfaction, Performance, and Efficiency are the most frequently investigated dimensions. Communication, however, remains underexplored with the human-human collaboration being the least investigated communication sub-dimension. This is inline with our findings on LLM-assistants risks on collaboration and further highlights the need to investigate the impact of LLM-assistants for both human-human and human-agent collaboration and communication. As LLMs continue to advance, particularly in reasoning, and multimodal understanding, these dynamics may further evolve. Future studies should therefore examine how improvements in model capability reshape the balance between productivity, collaboration, and quality.

**Recommendation 2.** Researchers should continue advancing multidimensional evaluations of developer productivity by systematically addressing underexplored *SPACE* dimensions, particularly Communication and Collaboration. While Satisfaction, Performance, and Efficiency are frequently assessed, the human-human and human-agent interaction aspects remain limited. Future studies should incorporate richer measures of team dynamics and communication patterns to better understand how LLM-assistants affect collaborative workflows. Priority areas for investigation include: (i) the effects of LLM adoption on pair programming and code review practices; (ii) developer well-being, occupational stress, and sustainable work practices; (iii) shifts in the allocation of developer time across tasks following LLM adoption; and (iv) the development and validation of instruments specifically designed for LLM-assisted development contexts.

**Accounting for confounding variables.** The variations in reported productivity outcomes across studies can often be attributed to confounding variables such as developer experience, task complexity, and domain context. These factors can impact the observed benefits and risks of LLM-assistants. For instance, novices may show higher immediate gains but also greater over-reliance and long-term skill erosion [PS12, PS20, PS39], while expert developers often achieve sustained efficiency through selective and critical use [PS11, PS14, PS19]. Likewise, LLMs tend to perform well on isolated, well-defined tasks but struggle in complex, context-rich projects where validation overhead offsets speed gains [PS9, PS14]. Current laboratory-based studies can have limited ecological validity. Future research should explicitly model and report these contextual variables to enhance fairer comparisons and more cumulative insight.

**Recommendation 3.** Researchers should systematically account for confounding variables that influence productivity outcomes, including developer expertise, task complexity, and domain context. Study designs should incorporate these as covariates, ensuring that effects attributed to LLM-assistants reflect genuine productivity changes rather than contextual bias. Explicitly reporting these variables will strengthen cross-study comparability and help build a context-aware evidence base. Essential methodological practices include: (i) stratifying analyses by experience level and reporting differential effects for novice versus expert developers; (ii) operationalizing task complexity along defined dimensions and examining its moderating role on LLM effectiveness; (iii) documenting organizational context to facilitate meta-analytic synthesis; and (iv) conducting replication studies across diverse populations, tools, and contexts to establish boundary conditions for reported findings.

## 9 Threats to Validity

This systematic review acknowledges several threats to the validity of its findings, which arise both from the methodology employed and the evolving nature of the primary evidence base.

## 9.1 Study Selection and Classification Rigor

**Study selection bias.** A key threat lies in the potential omission of relevant studies due to the inclusion and exclusion criteria defined in our protocol (see Section 3.1.1). Specifically, we exclude shorter papers, non-peer-reviewed work, and publications outside academic journals and conference proceedings. To mitigate this, all authors collaboratively agree on these criteria to ensure methodological rigor and quality control to enhance the reliability of the synthesized findings. Another threat relates to the challenge of identifying the human-centered studies within the large body of LLM research. Our initial search strings include terms such as “performance” or “efficiency”, which yielded results focused on the technical applications of LLMs rather than their impact on developer productivity. This issue stems from the broad scope of LLM4SE research [1], where such keywords frequently describe model behavior or algorithmic improvements rather than developer-centric outcomes. To mitigate this threat, three authors jointly review the selected control papers to ensure alignment with the inclusion criteria. We iteratively validate our search strings using control articles inspired by Zhang, Babar, and Tell [42] (see section 3.1.2), ensuring all control papers are correctly retrieved. Furthermore, we employ backward and forward snowballing to further enhance the coverage of relevant articles and reduce the risk of missing key studies.

**Bias and repeatability.** The open-ended nature of our research questions can introduce the threat of taking subjective decisions in the interpretation and selection of studies. To mitigate this, we adopt a multi-step validation process involving all co-authors. While the initial screening and data extraction were conducted by one author, the remaining authors were actively involved in selection validation and protocol design. The teams held regular weekly meetings for a period of 9 months to discuss the inclusion decisions and refine the selection and data extraction process. Additionally, we employ a conservative screening strategy during the full-text review, whereby any study with uncertain eligibility was retained for further assessment and independently reviewed by two senior co-authors to minimize the risk of excluding relevant studies.

**Classification rigor.** A potential threat lies in the mapping of study findings to the *SPACE* framework (see Section 7). Since the *SPACE* framework was not originally developed to represent productivity in human-LLM collaboration settings, assigning findings to specific sub-dimensions required interpretive decisions. This may have introduced subjective bias during data coding. To mitigate this, we adapted definitions from prior literature [19, 65] to better fit the context of our review, and discussed coding decisions collaboratively in team meetings.

## 9.2 Limitations of the Primary Evidence Base

**Formative and controlled studies.** A notable limitation inherited from the primary literature is the high proportion of formative (59%) and laboratory experiment (38%) studies (see section 5). While this emphasis provides strong internal validity (i.e., controlled effects), it can hinder the ecological validity of the findings. A number of reported productivity results largely reflect performance on isolated tasks and controlled environments. While these findings can overlook the complex realities of large-scale industry development, they are critical to building foundational knowledge.

**Methodological diversity.** Methodological diversity across the primary studies can hinder comparability. The literature exhibits a lack of standardized metrics, particularly for highly contested concepts like code quality and cognitive load. For example, studies employ diverse measures, ranging from cyclomatic complexity and code coverage to defect density and functional correctness, to evaluate code quality. While this diversity can hinder comparability, it can also support the triangulation of findings from different sources of evidence.

**Temporal relevance.** Given the rapid pace of progress in Generative AI, a key threat concerns the temporal completeness of the review. Our search and data extraction were conducted at the end of 2024, with 77% of the included studies published in the same year. As LLM capabilities, evaluation methods, and developer practices evolve, new findings may emerge quickly after our review period. To mitigate this limitation, we emphasize the

importance of transparent, replicable protocols and encourage periodic updates that can incorporate emerging empirical evidence in this fast-evolving field.

## 10 Conclusion

In this paper, we investigate LLM-assistants' impact on developer productivity. To achieve this, we systematically identify and analyze 39 peer-reviewed studies from their methodological strategies, evaluation practices, and the productivity dimensions these studies focus on. We synthesize reported benefits and risks, and apply established conceptual frameworks to map our findings and contextualize their broader implications.

Our analysis reveals a range of reported benefits, including reduced task initiation overhead, accelerated development, and support for code-adjacent tasks. At the same time, studies identify several risks, such as over-reliance on LLM-assistants especially affecting novice programmers, disruptions to developer flow, and reduced team communication or collaboration. Code quality, in particular, has a mixed outcome, with studies reporting both improvements and degradations depending on context, task design, and evaluation criteria. Our findings show that most studies (90%) consider multiple productivity dimensions. However, relatively few studies extend beyond three, and dimensions like communication and, more specifically, human-human collaboration remain underexplored.

Looking ahead, there is value in expanding the evidence base through team-based studies that capture the dynamic and socio-technical nature of software development. As LLM-assistants become more deeply embedded in everyday workflows, future research will play a critical role in understanding the multidimensional impact of LLM-assistants on developer productivity. To facilitate transparency and future work, we provide a publicly available replication package [20].

## 11 Acknowledgment

This work was supported by NSERC Discovery Grant RGPIN-2024-06511.

## References

- [1] Xinyi Hou et al. "Large language models for software engineering: A systematic literature review". In: *ACM Transactions on Software Engineering and Methodology* 33.8 (2024), pp. 1–79.
- [2] OpenAI. *GPT-4 Technical Report*. Accessed: 2025-06-12. 2023. URL: <https://openai.com/research/gpt-4>.
- [3] GitHub. *GitHub Copilot*. <https://github.com/features/copilot>. Accessed: 2025-06-12. 2021.
- [4] Alessio Buscemi. "A comparative study of code generation using chatgpt 3.5 across 10 programming languages". In: *arXiv preprint arXiv:2308.04477* (2023).
- [5] Xiaodong Gu et al. "On the effectiveness of large language models in domain-specific code generation". In: *ACM Transactions on Software Engineering and Methodology* 34.3 (2025), pp. 1–22.
- [6] Seohyun Kim et al. "Code prediction by feeding trees to transformers". In: *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE. 2021, pp. 150–162.
- [7] Yuwei Zhang et al. "PATCH: Empowering Large Language Model with Programmer-Intent Guidance and Collaborative-Behavior Simulation for Automatic Bug Fixing". In: *ACM Transactions on Software Engineering and Methodology* (2025).
- [8] Guoyang Weng and Artur Andrzejak. "Automatic bug fixing via deliberate problem solving with large language models". In: *2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE. 2023, pp. 34–36.
- [9] Runchu Tian et al. "Debugbench: Evaluating debugging capability of large language models". In: *arXiv preprint arXiv:2401.04621* (2024).

- [10] Shubhang Shekhar Dvivedi et al. “A comparative analysis of large language models for code documentation generation”. In: *Proceedings of the 1st ACM International Conference on AI-Powered Software*. 2024, pp. 65–73.
- [11] Maram Assi, Safwat Hassan, and Ying Zou. “LLM-Cure: LLM-based Competitor User Review Analysis for Feature Enhancement”. In: *ACM Trans. Softw. Eng. Methodol.* (June 2025). ISSN: 1049-331X. DOI: 10.1145/3744644. URL: <https://doi.org/10.1145/3744644>.
- [12] Junjie Wang et al. “Software testing with large language models: Survey, landscape, and vision”. In: *IEEE Transactions on Software Engineering* (2024).
- [13] Angela Fan et al. “Large language models for software engineering: Survey and open problems”. In: *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 2023, pp. 31–53.
- [14] Shantanu Mandal et al. “Large language models based automatic synthesis of software specifications”. In: *arXiv preprint arXiv:2304.09181* (2023).
- [15] Jules White et al. “Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design”. In: *Generative ai for effective software development*. Springer, 2024, pp. 71–108.
- [16] Christian Bird et al. “Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools”. In: *Queue* 20.6 (2022), pp. 35–57.
- [17] Emerson Murphy-Hill et al. “What predicts software developers’ productivity?” In: *IEEE Transactions on Software Engineering* 47.3 (2019), pp. 582–594.
- [18] Moritz Beller et al. “Mind the gap: on the relationship between automatically measured and self-reported productivity”. In: *IEEE Software* 38.5 (2020), pp. 24–31.
- [19] N Forsgren et al. *The SPACE of developer productivity: There’s more to it than you think*. *acmqueue* 19 (1), 20–48. 2021.
- [20] Amr Mohamed, Maram Assi, and Mariam Guizani. *The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Review and Mapping Study*. Supplemental Material. 2025. DOI: 10.5281/zenodo.18489222. URL: <https://zenodo.org/records/18489222>.
- [21] Frederick P Brooks Jr. *The mythical man-month: essays on software engineering*. Pearson Education, 1995.
- [22] Tom DeMarco and Tim Lister. *Peopleware: productive projects and teams*. Addison-Wesley, 2013.
- [23] Rebecca E Grinter, James D Herbsleb, and Dewayne E Perry. “The geography of coordination: Dealing with distance in R&D work”. In: *Proceedings of the 1999 ACM International Conference on Supporting Group Work*. 1999, pp. 306–315.
- [24] Audris Mockus, Roy T Fielding, and James D Herbsleb. “Two case studies of open source software development: Apache and Mozilla”. In: *ACM Transactions on Software Engineering and Methodology (TOSEM)* 11.3 (2002), pp. 309–346.
- [25] Stefan Wagner and Melanie Ruhe. “A systematic review of productivity factors in software development”. In: *arXiv preprint arXiv:1801.06475* (2018).
- [26] Daniel Graziotin et al. “What happens when software developers are (un) happy”. In: *Journal of Systems and Software* 140 (2018), pp. 32–47.
- [27] Kai Petersen. “Measuring and predicting software productivity: A systematic map and review”. In: *Information and Software Technology* 53.4 (2011), pp. 317–343.
- [28] Tony Savor et al. “Continuous deployment at Facebook and OANDA”. In: *Proceedings of the 38th International Conference on software engineering companion*. 2016, pp. 21–30.
- [29] Minghui Zhou and Audris Mockus. “Developer fluency: Achieving true mastery in software projects”. In: *Proceedings of the eighteenth ACM SIGSOFT international symposium on Foundations of software engineering*. 2010, pp. 137–146.

- [30] Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The science of lean software and devops: Building and scaling high performing technology organizations*. IT Revolution, 2018.
- [31] Abi Noda et al. “DevEX: What actually drives productivity?” In: *Communications of the ACM* 66.11 (2023), pp. 44–49.
- [32] Fabian Fagerholm and Jürgen Münch. “Developer experience: Concept and definition”. In: *2012 International Conference on Software and System Process (ICSSP)*. 2012, pp. 73–77. DOI: 10.1109/ICSSP.2012.6225984.
- [33] Darja Smite et al. “Changes in perceived productivity of software engineers during COVID-19 pandemic: The voice of evidence”. In: *Journal of Systems and Software* 186 (2022), p. 111197.
- [34] Lan Cheng et al. “What improves developer productivity at google? code quality”. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2022, pp. 1302–1313.
- [35] Anastasia Ruvimova et al. “An exploratory study of productivity perceptions in software teams”. In: *Proceedings of the 44th International Conference on Software Engineering*. 2022, pp. 99–111.
- [36] Nils Brede Moe et al. “Improving productivity through corporate hackathons: A multiple case study of two large-scale agile organizations”. In: *arXiv preprint arXiv:2112.05528* (2021).
- [37] Margaret-Anne Storey, Brian Houck, and Thomas Zimmermann. “How developers and managers define and trade productivity for quality”. In: *Proceedings of the 15th International Conference on Cooperative and Human Aspects of Software Engineering*. 2022, pp. 26–35.
- [38] Paloma Guenes et al. “Impostor phenomenon in software engineers”. In: *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society*. 2024, pp. 96–106.
- [39] Darja Šmite et al. “From forced Working-From-Home to voluntary working-from-anywhere: Two revolutions in telework”. In: *Journal of Systems and Software* 195 (2023), p. 111509.
- [40] Staffs Keele et al. *Guidelines for performing systematic literature reviews in software engineering*. Tech. rep. Technical report, ver. 2.3 ebse technical report. ebse, 2007.
- [41] Andrew MacFarlane, Tony Russell-Rose, and Farhad Shokraneh. “Search strategy formulation for systematic reviews: Issues, challenges and opportunities”. In: *Intelligent Systems with Applications* 15 (2022), p. 200091.
- [42] He Zhang, Muhammad Ali Babar, and Paolo Tell. “Identifying relevant studies in software engineering”. In: *Information and Software Technology* 53.6 (2011), pp. 625–637.
- [43] Xuetao Li et al. “Systematic literature review of commercial participation in open source software”. In: *ACM Transactions on Software Engineering and Methodology* 34.2 (2025), pp. 1–31.
- [44] Joonas Hämäläinen, Teerath Das, and Tommi Mikkonen. “A Systematic Literature Review of Multi-Label Learning in Software Engineering”. In: *ACM Transactions on Software Engineering and Methodology* (2024).
- [45] Sin Kit Lo et al. “A systematic literature review on federated machine learning: From a software engineering perspective”. In: *ACM Computing Surveys (CSUR)* 54.5 (2021), pp. 1–39.
- [46] Abdul Razzaq et al. “A Systematic Literature Review on the Influence of Enhanced Developer Experience on Developers’ Productivity: Factors, Practices, and Recommendations”. In: *ACM Computing Surveys* 57.1 (2024), pp. 1–46.
- [47] Matthew J Page et al. “The PRISMA 2020 statement: an updated guideline for reporting systematic reviews”. In: *BMJ* 372 (2021). DOI: 10.1136/bmj.n71. eprint: <https://www.bmj.com/content/372/bmj.n71.full.pdf>. URL: <https://www.bmj.com/content/372/bmj.n71>.
- [48] Valentina Lenarduzzi et al. “A systematic literature review on technical debt prioritization: Strategies, processes, factors, and tools”. In: *Journal of Systems and Software* 171 (2021), p. 110827.

- [49] Vahid Garousi and Mika V Mäntylä. “A systematic literature review of literature reviews in software testing”. In: *Information and Software Technology* 80 (2016), pp. 195–216.
- [50] Klaas-Jan Stol and Brian Fitzgerald. “The ABC of software engineering research”. In: *ACM Transactions on Software Engineering and Methodology (TOSEM)* 27.3 (2018), pp. 1–51.
- [51] Robert L. Glass, Iris Vessey, and Venkataraman Ramesh. “Research in software engineering: an analysis of the literature”. In: *Information and Software Technology* 44.8 (2002), pp. 491–506.
- [52] H Rex Hartson, Terence S Andre, and Robert C Williges. “Criteria for evaluating usability evaluation methods”. In: *International journal of human-computer interaction* 13.4 (2001), pp. 373–410.
- [53] Sandra G Hart and Lowell E Staveland. “Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research”. In: *Advances in psychology*. Vol. 52. Elsevier, 1988, pp. 139–183.
- [54] Patrícia Silva. “Davis’ technology acceptance model (TAM)(1989)”. In: *Information seeking behavior and technology adoption: Theories and trends* (2015), pp. 205–219.
- [55] Deborah R Compeau and Christopher A Higgins. “Computer self-efficacy: Development of a measure and initial test”. In: *MIS quarterly* (1995), pp. 189–211.
- [56] Igor Steinmacher et al. “Overcoming open source project entry barriers with a portal for newcomers”. In: *Proceedings of the 38th International Conference on Software Engineering*. 2016, pp. 273–284.
- [57] James A Russell. “A circumplex model of affect.” In: *Journal of personality and social psychology* 39.6 (1980), p. 1161.
- [58] Lucian José Gonçalves, Kleinner Farias, and Bruno C da Silva. “Measuring the cognitive load of software developers: An extended Systematic Mapping Study”. In: *Information and Software Technology* 136 (2021), p. 106563.
- [59] Brittany Johnson, Thomas Zimmermann, and Christian Bird. “The effect of work environments on productivity and satisfaction of software engineers”. In: *IEEE Transactions on Software Engineering* 47.4 (2019), pp. 736–757.
- [60] Capers Jones. “Software metrics: good, bad and missing”. In: *Computer* 27.9 (1994), pp. 98–100.
- [61] Prem Devanbu et al. “Analytical and empirical evaluation of software reuse metrics”. In: *Proceedings of IEEE 18th International Conference on Software Engineering*. IEEE. 1996, pp. 189–199.
- [62] Shraddha Barke, Michael B James, and Nadia Polikarpova. “Grounded copilot: How programmers interact with code-generating models”. In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1 (2023), pp. 85–111.
- [63] Raja Parasuraman and Dietrich H Manzey. “Complacency and bias in human use of automation: An attentional integration”. In: *Human factors* 52.3 (2010), pp. 381–410.
- [64] Google. *DevOps Research and Assessment: 2022 State of DevOps Report*. <https://services.google.com/fh/files/misc/state-of-devops-2021.pdf>. [Online; accessed 12-Oct-2025]. Dec. 2022.
- [65] Samarth Sikand et al. “How much SPACE do metrics have in GenAI assisted software development?” In: *Proceedings of the 17th Innovations in Software Engineering Conference*. 2024, pp. 1–5.
- [66] Ketai Qiu et al. “From today’s code to tomorrow’s symphony: The AI transformation of developer’s routine by 2030”. In: *ACM Transactions on Software Engineering and Methodology* 34.5 (2025), pp. 1–17.
- [67] Silvia Abrahão et al. “Software Engineering by and for Humans in an AI Era”. In: *ACM Transactions on Software Engineering and Methodology* 34.5 (2025), pp. 1–46.
- [68] Marshall McLuhan. “Laws of the Media”. In: *ETC: A Review of General Semantics* 34.2 (1977). Retrieved from JSTOR, pp. 173–179. URL: <http://www.jstor.org/stable/42575246>.
- [69] Daniel Fontanet Losquiño and Tomas Urdell. “Why do developers struggle with documentation while excelling at programming”. B.S. thesis. Universitat Politècnica de Catalunya, 2014.
- [70] Auste Simkute et al. “Ironies of generative AI: understanding and mitigating productivity loss in Human-AI interaction”. In: *International Journal of Human-Computer Interaction* 41.5 (2025), pp. 2898–2919.

## Primary Studies

- [PS1] Frank F Xu, Bogdan Vasilescu, and Graham Neubig. “In-ide code generation from natural language: Promise and challenges”. In: *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31.2 (2022), pp. 1–47.
- [PS2] Justin D Weisz et al. “Better together? an evaluation of ai-supported code translation”. In: *Proceedings of the 27th International Conference on Intelligent User Interfaces*. 2022, pp. 369–391.
- [PS3] Sandeep Kaur Kuttal et al. “Trade-offs for substituting a human with an agent in a pair programming context: the good, the bad, and the ugly”. In: *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 2021, pp. 1–20.
- [PS4] Olivia B Newton et al. “EveryBOTy counts: Examining human–machine teams in open source software development”. In: *Topics in Cognitive Science* 16.3 (2024), pp. 450–484.
- [PS5] Vijayaraghavan Murali et al. “AI-assisted Code Authoring at Scale: Fine-tuning, deploying, and mixed methods evaluation”. In: *Proceedings of the ACM on Software Engineering* 1.FSE (2024), pp. 1066–1085.
- [PS6] Shaokang Jiang and Michael Coblenz. “An Analysis of the Costs and Benefits of Autocomplete in IDEs”. In: *Proceedings of the ACM on Software Engineering* 1.FSE (2024), pp. 1284–1306.
- [PS7] Ranim Khojah et al. “Beyond code generation: An observational study of chatgpt usage in software engineering practice”. In: *Proceedings of the ACM on Software Engineering* 1.FSE (2024), pp. 1819–1840.
- [PS8] Wei Wang et al. “Rocks coding, not development: A human-centric, experimental evaluation of LLM-supported SE tasks”. In: *Proceedings of the ACM on Software Engineering* 1.FSE (2024), pp. 699–721.
- [PS9] Daniel Russo. “Navigating the complexity of generative ai adoption in software engineering”. In: *ACM Transactions on Software Engineering and Methodology* 33.5 (2024), pp. 1–50.
- [PS10] Zhensu Sun et al. “Don’t Complete It! Preventing Unhelpful Code Completion for Productive and Sustainable Neural Code Completion Systems”. In: *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE. 2023, pp. 324–325.
- [PS11] Jenny T Liang, Chenyang Yang, and Brad A Myers. “A large-scale survey on the usability of ai programming assistants: Successes and challenges”. In: *Proceedings of the 46th IEEE/ACM international conference on software engineering*. 2024, pp. 1–13.
- [PS12] Rudrajit Choudhuri et al. “How far are we? the triumphs and trials of generative ai in learning software engineering”. In: *Proceedings of the IEEE/ACM 46th international conference on software engineering*. 2024, pp. 1–13.
- [PS13] Daye Nam et al. “Using an llm to help with code understanding”. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 2024, pp. 1–13.
- [PS14] Nicole Davila et al. “An industry case study on adoption of ai-based programming assistants”. In: *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. 2024, pp. 92–102.
- [PS15] Ebtesam Al Haque et al. “The Evolution of Information Seeking in Software Development: Understanding the Role and Impact of AI Assistants”. In: *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 2025, pp. 1494–1502.
- [PS16] Albert Ziegler et al. “Productivity assessment of neural code completion”. In: *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 2022, pp. 21–29.
- [PS17] Priyam Sahoo et al. “Ansible Lightspeed: A Code Generation Service for IT Automation”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 2024, pp. 2148–2158.
- [PS18] Jinrun Liu et al. “Chatgpt vs. stack overflow: An exploratory comparison of programming assistance tools”. In: *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*. IEEE. 2023, pp. 364–373.

- [PS19] Mohammad Amin Kuhail et al. ““Will I be replaced?” Assessing ChatGPT’s effect on software development and programmer perceptions of AI tools”. In: *Science of Computer Programming* 235 (2024), p. 103111.
- [PS20] Simone Mezzaro, Alessio Gambi, and Gordon Fraser. “An empirical study on how large language models impact software testing learning”. In: *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 2024, pp. 555–564.
- [PS21] Muhammad Waseem et al. “ChatGPT as a software development bot: a project-based study”. In: *19th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)* (2023).
- [PS22] Hussein Mozannar et al. “Reading between the lines: Modeling user behavior and costs in AI-assisted programming”. In: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 2024, pp. 1–16.
- [PS23] Crystal Qian and James Wexler. “Take it, leave it, or fix it: measuring productivity and trust in human-AI collaboration”. In: *Proceedings of the 29th International Conference on Intelligent User Interfaces*. 2024, pp. 370–384.
- [PS24] Thomas Weber et al. “Significant productivity gains through programming with large language models”. In: *Proceedings of the ACM on Human-Computer Interaction* 8.EICS (2024), pp. 1–29.
- [PS25] Helena Vasconcelos et al. “Generation Probabilities Are Not Enough: Uncertainty Highlighting in AI Code Completions”. In: *ACM Transactions on Computer-Human Interaction* (2024).
- [PS26] Jacques Bughin. “The role of firm AI capabilities in generative AI-pair coding”. In: *Journal of Decision Systems* (2024), pp. 1–22.
- [PS27] Danie Smit et al. “The impact of GitHub Copilot on developer productivity from a software engineering body of knowledge perspective”. In: (2024).
- [PS28] Asha Rajbhoj et al. “Accelerating software development using generative ai: Chatgpt case study”. In: *Proceedings of the 17th innovations in software engineering conference*. 2024, pp. 1–11.
- [PS29] Shreyas Pangavhane et al. “AI-augmented software development: Boosting efficiency and quality”. In: *2024 International Conference on Decision Aid Sciences and Applications (DASA)*. IEEE. 2024, pp. 1–5.
- [PS30] Ojelanki Ngwenyama, Nada Kanita, and Frantz Rowe. “Can Generative AI Contribute to both productivity gains and human Flourishing, and in fine satisfaction at work? Research on GitHub Copilot use in Software Development”. In: (2025).
- [PS31] Kathrin Komp-Leukkunen. “How ChatGPT shapes the future labour market situation of software engineers: A Finnish Delphi study”. In: *Futures* 160 (2024), p. 103382.
- [PS32] Jacques Bughin. “What drives the corporate payoffs of using generative artificial intelligence?” In: *Structural Change and Economic Dynamics* 71 (2024), pp. 658–668.
- [PS33] Wendy Mendes, Samara Souza, and Cleidson De Souza. ““You’re on a bicycle with a little motor”: Benefits and Challenges of Using AI Code Assistants”. In: *Proceedings of the 2024 IEEE/ACM 17th International Conference on Cooperative and Human Aspects of Software Engineering*. 2024, pp. 144–152.
- [PS34] Takura Mbizo et al. “Cautious optimism: The influence of generative AI tools in software development projects”. In: *Annual conference of south african institute of computer scientists and information technologists*. Springer. 2024, pp. 361–373.
- [PS35] Gustavo Pinto et al. “Developer Experiences with a Contextualized AI Coding Assistant: Usability, Expectations, and Outcomes”. In: *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*. 2024, pp. 81–91.
- [PS36] Mariana Coutinho et al. “The role of generative ai in software development productivity: A pilot case study”. In: *Proceedings of the 1st ACM International Conference on AI-Powered Software*. 2024, pp. 131–138.

- [PS37] Tianyi Chen. “The Impact of AI-Pair Programmers on Code Quality and Developer Satisfaction: Evidence from TiMi studio”. In: *Proceedings of the 2024 International Conference on Generative Artificial Intelligence and Information Security*. 2024, pp. 201–205.
- [PS38] Nicholas Gardella, Raymond Pettit, and Sara L Riggs. “Performance, Workload, Emotion, and Self-Efficacy of Novice Programmers Using AI Code Generation”. In: *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*. 2024, pp. 290–296.
- [PS39] Eduard Frankford et al. “Ai-tutoring in software engineering education”. In: *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training*. 2024, pp. 309–319.

Received 2 July 2025; revised 4 Feb 2026; accepted 17 March 2026